

Cryptography — Study Guide for Exam 1

1 Part I — Formal Foundations of Security

1.1 Encryption Schemes and Correctness

A private-key encryption scheme is a triple of algorithms

$$\Pi = (\text{Gen}, \text{Enc}, \text{Dec}).$$

With security parameter λ :

$$k \leftarrow \text{Gen}(1^\lambda), \quad c \leftarrow \text{Enc}_k(m), \quad m' := \text{Dec}_k(c).$$

- Gen is probabilistic;
- Enc may be probabilistic or use a nonce/IV;
- Dec is usually deterministic;
- **correctness** requires, for every valid message,

$$\Pr[\text{Dec}_k(\text{Enc}_k(m)) = m] = 1,$$

where the probability accounts for all the randomness of Gen and Enc.

Security and correctness are different properties. Correctness says that the recipient recovers the message; security says what an adversary cannot learn or produce.

1.2 Perfect Secrecy

Consider random variables M , K and $C = \text{Enc}_K(M)$. Before observing C , the probability $\Pr[M = m]$ represents the *a priori* knowledge about the message. After observing a ciphertext c , that knowledge becomes $\Pr[M = m \mid C = c]$. There is perfect secrecy when, for every message distribution, every message with positive probability and every ciphertext that can occur,

$$\Pr[M = m \mid C = c] = \Pr[M = m].$$

Therefore, not even an adversary with unlimited computational power updates its belief about M upon seeing C . This is a statement of statistical independence, not of computational difficulty.

The two central characterizations are:

$$\Pr[M = m \mid C = c] = \Pr[M = m]$$

and, for all $m, m' \in \mathcal{M}$ and $c \in \mathcal{C}$,

$$\Pr[\text{Enc}_K(m) = c] = \Pr[\text{Enc}_K(m') = c].$$

Lemma 2.5 proves that these characterizations are equivalent. To understand the less obvious direction, suppose that the ciphertext distribution is identical for any message. Fixing c , there exists a value p_c such that

$$\Pr[C = c \mid M = m] = p_c$$

for every m . By the law of total probability,

$$\Pr[C = c] = \sum_{m'} \Pr[C = c \mid M = m'] \Pr[M = m'] = p_c.$$

Applying Bayes,

$$\Pr[M = m \mid C = c] = \frac{\Pr[C = c \mid M = m] \Pr[M = m]}{\Pr[C = c]} = \frac{p_c \Pr[M = m]}{p_c} = \Pr[M = m].$$

In the converse direction, choose the uniform distribution over $\mathcal{M}$. If observing c does not change the probability of any message, then $\Pr[C = c \mid M = m]$ cannot depend on m ; if it did, Bayes would change the posterior distribution.

1.2.1 One-Time Pad as a Complete Example

For messages of ℓ bits, choose $K \xleftarrow{\$} \{0, 1\}^\ell$ and define

$$C = M \oplus K, \quad M = C \oplus K.$$

Fix any $m, c \in \{0, 1\}^\ell$. There is exactly one key capable of mapping m to c , namely $k = m \oplus c$. Since all 2^ℓ keys are equally likely,

$$\Pr[\text{Enc}_K(m) = c] = 2^{-\ell},$$

independently of m . By Lemma 2.5, the OTP has perfect secrecy.

The conditions are not optional: the key must be uniform, as long as the message, kept secret and used only once. If the same key encrypts m and m' , then

$$c \oplus c' = m \oplus m',$$

and the perfect secrecy for the set of the two messages disappears.

For perfectly secret schemes with finite spaces and correctness, the bound

$$|\mathcal{K}| \geq |\mathcal{M}|.$$

holds. The intuition is that, fixing a possible ciphertext c , each message needs at least one key that maps it to c ; by correctness, the same key cannot decrypt c into two different messages. Hence at least as many keys as messages are needed. It is this cost that motivates the relaxation to computational security.

1.2.2 Lemma 2.5 versus Definition 2.6

Lemma 2.5 directly compares the ciphertext distributions produced by any two messages. No ciphertext can favor one message.

Definition 2.6 presents the same goal as a game:

1. the adversary chooses $m_0, m_1 \in \mathcal{M}$;
2. the oracle samples $b \xleftarrow{\$} \{0, 1\}$;
3. it returns $c \leftarrow \text{Enc}_K(m_b)$;
4. the adversary produces b' .

Perfect secrecy requires

$$\Pr[b' = b] = \frac{1}{2}$$

even for adversaries with no computational limit.

In Definition 2.6 of the book, equality of length is not written separately; it is automatic in the examples where $\mathcal{M}$ contains only messages of a fixed length. In Definition 3.8, which admits variable lengths, the restriction $|m_0| = |m_1|$ appears explicitly so that the length does not reveal the bit b .

Advantage of the game-based formulation: it generalizes easily. It suffices to change the adversary's powers, the available oracles and the winning condition. This is how EAV, CPA, CCA, MAC security and signature security arise.

1.3 Concrete and Asymptotic Security

There are two ways to state that a scheme is secure. The concrete approach fixes explicit bounds on time and success; the asymptotic approach views both as functions of a security parameter λ and requires a negligible advantage.

1.3.1 Concrete Approach

A construction can be declared (t, ε) -secure when no adversary running in time at most t achieves **advantage** greater than ε . In a game whose baseline is $1/2$, this means success at most $1/2 + \varepsilon$; it does not mean success at most ε .

This approach answers engineering questions: how many operations are needed and what residual risk is accepted?

1.3.2 Asymptotic Approach

The adversary is a probabilistic polynomial-time algorithm, abbreviated **PPT**. The security parameter is λ , and the additional success allowed must be a negligible function of λ .

A function $\mu : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is **negligible** if

$$\forall c > 0 \exists N \forall \lambda \geq N : \quad \mu(\lambda) < \frac{1}{\lambda^c}.$$

Equivalently, for every positive polynomial p , there exists N such that

$$\lambda \geq N \implies \mu(\lambda) < \frac{1}{p(\lambda)}.$$

Examples:

$$2^{-\lambda}, \quad 2^{-\sqrt{\lambda}} \quad \text{are negligible;}$$

$$\frac{1}{\lambda}, \quad \frac{1}{\lambda^{100}}, \quad \frac{1}{\log \lambda} \quad \text{are not negligible.}$$

The reason $1/\lambda^{100}$ is not negligible is the order of the quantifiers: the function must beat the inverse of **every** polynomial, not merely of some polynomial.

1.3.3 Sum of Negligible Functions

If μ_1 and μ_2 are negligible, then so is $\mu_1 + \mu_2$. Fix $c > 0$. Since each function is negligible, for sufficiently large λ :

$$\mu_1(\lambda) < \frac{1}{2\lambda^c}, \quad \mu_2(\lambda) < \frac{1}{2\lambda^c}.$$

Therefore:

$$\mu_1(\lambda) + \mu_2(\lambda) < \frac{1}{2\lambda^c} + \frac{1}{2\lambda^c} = \frac{1}{\lambda^c}.$$

It also holds that a polynomial times a negligible function remains negligible.

1.4 Definition 3.8 versus Definition 2.6

Both use an experiment with two messages. The difference is the type of guarantee:

Property	Adversary	Messages	Winning probability
Def. 2.6: perfect secrecy	unlimited	in the fixed space $\mathcal{M}$	exactly $1/2$
Def. 3.8: EAV	PPT, receives 1^λ	same length	at most $1/2 + \text{negl}(\lambda)$

Definition 3.8 introduces the security parameter and views both time and success as functions of λ . It is strictly weaker: every perfectly secret scheme is EAV-secure, but the pseudo-OTP will be EAV-secure with a key shorter than the message and therefore cannot have perfect secrecy.

The advantage of $\mathcal{A}$ can be written as

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{eav}}(\lambda) = \left| \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(\lambda) = 1] - \frac{1}{2} \right|.$$

Computational security requires negligible advantage for every PPT adversary.

1.5 Semantic Security

Intuition: observing the ciphertext must not allow an efficient adversary to compute any information about the message that it could not obtain without the ciphertext.

Examples of forbidden information:

- a specific bit of m ;
- a predicate $f(m)$;
- whether the value is above a threshold;
- whether two parts of the message are equal.

Semantic security and indistinguishability of encryptions are equivalent definitions under the appropriate formulation. In the proof, the important explanation is:

If the adversary distinguishes encryptions of m_0 and m_1 , choose a property that has different values on the two messages. If it learns a property of the message, choose two distributions/messages in which that property allows distinguishing.

1.5.1 The Semantic Experiment, Without Hiding the Quantifiers

Consider a message distribution M that can be sampled in polynomial time, an efficient function f that represents the desired information and, optionally, a function h that represents auxiliary information already known. The real adversary receives

$$C \leftarrow \text{Enc}_K(M) \quad \text{e} \quad h(M)$$

and tries to compute $f(M)$. Semantic security states that there exists a PPT simulator $\mathcal{S}$, which receives only $h(M)$ and the message length, whose success is at most a negligible amount below the success of the real adversary:

$$\Pr[\mathcal{A}(C, h(M)) = f(M)] - \Pr[\mathcal{S}(1^\lambda, |M|, h(M)) = f(M)] \leq \text{negl}(\lambda).$$

The simulator does not need to guess $f(M)$ perfectly. It needs to show that the ciphertext gives no significant advantage over what would already be possible without seeing it. For example, if 90% of the messages in the distribution start with zero, guessing zero already succeeds 90% of the time; the definition does not require hiding what the distribution itself reveals.

This observation eliminates a common confusion: “the attacker got something right” does not prove insecurity. It is necessary to compare the success against a baseline without the ciphertext.

1.5.2 From Indistinguishability to the Protection of Predicates

Suppose an algorithm extracts from the ciphertext a predicate $f(m)$ better than the baseline. Choose two messages m_0, m_1 for which $f(m_0) \neq f(m_1)$. Given an encryption of one of them, run the extractor and use the recovered value to guess the bit b . An advantage in extraction becomes an advantage in distinguishing.

Conversely, if $\mathcal{A}$ distinguishes encryptions of m_0 and m_1 , define the desired information as “which of the two messages was chosen”. The distinguisher is already an algorithm that learns this information. The formal proof needs to handle distributions and auxiliary information, but this is the logical core of the equivalence.

1.6 Pseudorandom Generators

A PRG is an efficient deterministic algorithm

$$G : \{0, 1\}^\lambda \longrightarrow \{0, 1\}^{\ell(\lambda)}, \quad \ell(\lambda) > \lambda.$$

For a uniform seed $s \xleftarrow{\$} \{0, 1\}^\lambda$, it is required that $G(s)$ be computationally indistinguishable from $r \xleftarrow{\$} \{0, 1\}^{\ell(\lambda)}$:

$$|\Pr[D(G(s)) = 1] - \Pr[D(r) = 1]| \leq \text{negl}(\lambda)$$

for every PPT distinguisher D .

1.6.1 The Distributions Are Really Different

If $\ell(\lambda) = 2\lambda$, then:

$$|\text{Im}(G)| \leq 2^\lambda \quad \text{e} \quad |\{0, 1\}^{2\lambda}| = 2^{2\lambda}.$$

The uniform distribution falls into the image of G with probability at most

$$\frac{2^\lambda}{2^{2\lambda}} = 2^{-\lambda}.$$

Meanwhile an output of G falls into this image with probability 1. Hence the statistical distance between the distributions is at least $1 - 2^{-\lambda}$, that is, almost maximal.

An unlimited algorithm could enumerate all seeds and distinguish almost always. Security states only that no **PPT** algorithm can perform this test. Therefore, the distributions are statistically very different, although computationally indistinguishable.

1.7 Pseudo-One-Time Pad

With $|m| = \ell(\lambda)$:

$$\text{Enc}_k(m) = m \oplus G(k), \quad \text{Dec}_k(c) = c \oplus G(k).$$

The key has only λ bits, but it encrypts larger messages. There is no contradiction with the OTP bound: the pseudo-OTP offers computational security, not perfect secrecy.

This construction is secure in the EAV experiment of **a single encryption**. Reusing the same key for two messages reuses the pad $G(k)$ and reveals the XOR of the plaintexts. For multiple messages, it is necessary to introduce randomness, state or a nonce so that different pads are produced.

1.8 Proofs by Reduction

A reduction has the form:

$$\text{adversary } \mathcal{A} \text{ breaks } \Pi \implies \text{algorithm } \mathcal{B}^{\mathcal{A}} \text{ breaks the assumption } H.$$

Mandatory structure:

1. **Assumption:** there exists $\mathcal{A}$ with non-negligible advantage against Π .
2. **Construction:** $\mathcal{B}$ receives an instance of problem H and uses $\mathcal{A}$ as a subroutine.
3. **Simulation:** the view given to $\mathcal{A}$ must have the correct distribution.
4. **Case analysis:** when the input to $\mathcal{B}$ is real and when it is random/fake.
5. **Advantage relation:** the advantage of $\mathcal{B}$ is computed from the advantage of $\mathcal{A}$.
6. **Contradiction:** this violates the security assumption of H .

In the pseudo-OTP, the reducer receives w , which is $G(k)$ or uniform. It chooses b , hands $c = w \oplus m_b$ to the adversary and answers “PRG” when $b' = b$. If w is uniform, the game is OTP; if $w = G(k)$, the game is the real scheme.

1.8.1 Reduction for the Pseudo-OTP, with the Advantage Computed

Let $\mathcal{A}$ be an EAV adversary against the pseudo-OTP. The distinguisher $\mathcal{B}$ for the PRG receives a string $w \in \{0, 1\}^{\ell(\lambda)}$ and executes:

1. obtains from $\mathcal{A}$ two messages m_0, m_1 of the same length;
2. samples $b \xleftarrow{\$} \{0, 1\}$;
3. forms $c = m_b \oplus w$ and hands c to $\mathcal{A}$;
4. receives b' and returns 1 if and only if $b' = b$.

There are two worlds.

PRG world: if $w = G(k)$, the simulation is exactly the real encryption. Therefore,

$$\Pr[\mathcal{B}(G(k)) = 1] = \Pr[b' = b \mid \text{pseudo-OTP}].$$

Uniform world: if w is uniform, $m_b \oplus w$ is also uniform and independent of b . This is a fresh OTP for a single message, hence

$$\Pr[\mathcal{B}(w) = 1 \mid w \leftarrow U_\ell] = \frac{1}{2}.$$

Consequently,

$$\text{Adv}_{G, \mathcal{B}}^{\text{prg}} = \left| \Pr[b' = b \mid \text{pseudo-OTP}] - \frac{1}{2} \right| = \text{Adv}_{\Pi, \mathcal{A}}^{\text{eav}}.$$

If the advantage of $\mathcal{A}$ were non-negligible, the advantage of $\mathcal{B}$ would also be, contradicting the security of the PRG. Note the three details that make the proof valid: $\mathcal{B}$ is efficient if $\mathcal{A}$ is efficient; the simulation of the real world is perfect; and in the uniform world the probability is exactly $1/2$.

1.8.2 Hybrids

When a construction uses many pseudorandom objects, one usually swaps one object at a time. If H_0 is the real game, H_t the ideal game and $H_1, \dots, H_{t-1}$ are hybrids, then

$$|\Pr[H_0 = 1] - \Pr[H_t = 1]| \leq \sum_{i=0}^{t-1} |\Pr[H_i = 1] - \Pr[H_{i+1} = 1]|.$$

If the total difference is non-negligible and t is polynomial, then at least one adjacent swap has non-negligible difference. That swap points exactly to where to embed the reduction's challenge.

1.9 EAV and CPA

EAV (eavesdropping) and CPA (chosen-plaintext attack) are the two basic indistinguishability notions. They differ only in the power granted to the adversary: EAV gives a single ciphertext, CPA adds an encryption oracle.

1.9.1 EAV: A Passive Observation

The adversary chooses m_0, m_1 , receives an encryption of m_b and tries to find out b .

1.9.2 CPA: Chosen Plaintext

Beyond the challenge, the adversary queries an encryption oracle:

$$m \mapsto \text{Enc}_k(m).$$

A deterministic cipher loses immediately: the adversary asks for $\text{Enc}_k(m_0)$ and compares with the challenge.

The full game has two query phases:

1. $k \leftarrow \text{Gen}(1^\lambda)$;
2. $\mathcal{A}$ queries $\text{Enc}_k(\cdot)$ as many polynomial times as it wants;
3. $\mathcal{A}$ sends m_0, m_1 with $|m_0| = |m_1|$;
4. the challenger samples $b \leftarrow \{0, 1\}$ and returns $c^* \leftarrow \text{Enc}_k(m_b)$;
5. $\mathcal{A}$ keeps querying the oracle and, at the end, returns b' .

The advantage is

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{cpa}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

Equality of lengths prevents a trivial win via the ciphertext length. The second phase prevents a proof that works only when the attacker stops interacting after the challenge.

1.9.3 Explicit Attack on a Deterministic Cipher

Choose $m_0 \neq m_1$ of the same length. Before the challenge, query

$$c_0 \leftarrow \text{Enc}_k(m_0).$$

Upon receiving $c^* = \text{Enc}_k(m_b)$, answer $b' = 0$ if $c^* = c_0$ and $b' = 1$ otherwise. Since encryption is deterministic and correct, encrypting distinct messages cannot produce the same ciphertext: if it did, decryption of the same value would have to return two messages. Hence the attack always succeeds:

$$\Pr[b' = b] = 1, \quad \text{Adv}^{\text{cpa}} = \frac{1}{2}.$$

1.9.4 PRF and PRP: The Abstraction Behind the Modes

A pseudorandom function family is indexed by a key:

$$F_k : \{0, 1\}^n \longrightarrow \{0, 1\}^m.$$

The adversary, with oracle access, cannot distinguish F_k from a truly random function chosen among all functions with that domain and codomain. A random function can produce collisions.

A pseudorandom permutation has equal domain and codomain and each P_k is bijective:

$$P_k : \{0, 1\}^n \longrightarrow \{0, 1\}^n.$$

It must be indistinguishable from a uniform permutation. Block ciphers such as AES are modeled as PRPs; for each key, decryption is the inverse permutation. The difference is observable after enough queries: a random function may collide, a permutation never does. For a number of queries much smaller than $2^{n/2}$, this difference is small, which explains why a PRP can often replace a PRF in analyses with a birthday-type loss.

Consequences:

- CPA requires randomness, a unique nonce or secure state;
- the challenge messages must have the same length;
- the adversary can choose queries adaptively;
- CPA confidentiality does not, by itself, provide integrity.

2 Part II — Symmetric Cryptography

2.1 Stream cipher versus block cipher

A **stream cipher** produces a pseudorandom keystream:

$$z_1 z_2 \dots := G(k, \text{nonce}), \quad c = m \oplus z.$$

Reusing the same keystream on two messages produces

$$c \oplus c' = (m \oplus z) \oplus (m' \oplus z) = m \oplus m',$$

eliminating the key from the computation.

A **block cipher** is a family of key-indexed permutations:

$$E : \mathcal{K} \times \{0, 1\}^n \rightarrow \{0, 1\}^n,$$

with an efficient inverse $D_k = E_k^{-1}$. Ideally, E_k behaves like a random permutation to anyone who does not know k : a PRP.

Concept	Input/output	Central property
PRG	short seed $\rightarrow$ long string	output looks uniform
PRF	fixed arbitrary input $\rightarrow$ output	function looks random
PRP/block cipher	block $\rightarrow$ block of the same size	permutation looks random

2.2 Feistel network

Split the block into (L_0, R_0) . In round i :

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F_i(R_{i-1}).$$

The function F_i need not be invertible. To undo the round:

$$R_{i-1} = L_i, \quad L_{i-1} = R_i \oplus F_i(L_i).$$

This works because $x \oplus y \oplus y = x$. A four-round network uses the subkeys k_1, k_2, k_3, k_4 on the way in and k_4, k_3, k_2, k_1 on the way back, keeping the same structure.

What to explain on the exam: Feistel creates an invertible permutation even when the round function is not invertible.

2.2.1 Full example with four rounds

Use 4-bit halves and, only to make the computation visible, define

$$F_{k_i}(R) = R \oplus k_i.$$

Take

$$(L_0, R_0) = (\mathbf{A}, \mathbf{3}), \quad (k_1, k_2, k_3, k_4) = (\mathbf{5}, \mathbf{C}, \mathbf{6}, \mathbf{9}).$$

Each row applies $L_i = R_{i-1}$ and $R_i = L_{i-1} \oplus (R_{i-1} \oplus k_i)$:

i	L_i	R_i	computation of R_i
0	A	3	input
1	3	C	$\mathbf{A} \oplus (\mathbf{3} \oplus \mathbf{5}) = \mathbf{C}$
2	C	3	$\mathbf{3} \oplus (\mathbf{C} \oplus \mathbf{C}) = \mathbf{3}$
3	3	9	$\mathbf{C} \oplus (\mathbf{3} \oplus \mathbf{6}) = \mathbf{9}$

i	L_i	R_i	computation of R_i
4	9	3	$3 \oplus (9 \oplus 9) = 3$

The result is $(L_4, R_4) = (9, 3)$. To go back one round, one does not compute F^{-1} . Instead one uses

$$R_{i-1} = L_i, \quad L_{i-1} = R_i \oplus F_{k_i}(L_i).$$

From the output, with $k_4 = 9$:

$$R_3 = 9, \quad L_3 = 3 \oplus (9 \oplus 9) = 3.$$

Repeating with k_3, k_2, k_1 , one recovers $(A, 3)$. The example also shows why the order of the subkeys must be reversed.

A few rounds are not enough for security. In one round, the left half of the output is literally the right half of the input; in two rounds there are still simple relations between input and output. The Luby–Rackoff result provides the theoretical foundation: three rounds with independent PRFs build a PRP and four build a strong PRP, under the appropriate definitions. This does not mean that a practical cipher should use only four rounds; DES uses sixteen because real functions, security margins, and cryptanalytic attacks demand more care.

2.3 DES and TripleDES

DES:

- block of 64 bits;
- nominal key of 64 bits, but only 56 effective bits;
- Feistel network with 16 rounds;
- main practical weakness: exhaustive search over 2^{56} ;
- a 64-bit block is also small for large data volumes.

DES remains important for understanding the history of cipher design: it was the first widely adopted civilian standard, popularized Feistel networks, motivated the public development of differential and linear cryptanalysis, and withstood decades of structural analysis. Its practical obsolescence does not make its architecture irrelevant; it shows, above all, that a cipher can have a well-studied structure and still fail because the key and the block became small.

TripleDES applies DES three times, typically in EDE:

$$c = E_{k_3}(D_{k_2}(E_{k_1}(m))).$$

EDE preserved compatibility with DES when the three keys coincided. 3DES increased security, but it is slow and keeps the block small. Its importance is historical: it showed how to extend the life of a primitive without redesigning the entire infrastructure.

2.3.1 Why 3DES does not simply deliver 168 bits

With two keys, the EDE construction is

$$C = E_{k_1}(D_{k_2}(E_{k_1}(M))).$$

With three keys there are 168 nominal bits, but *meet-in-the-middle* attacks and their variants reduce the effective margin. The meet-in-the-middle idea for a double encryption $C = E_{k_2}(E_{k_1}(M))$ is:

1. compute and store $E_{k_1}(M)$ for all keys k_1 ;
2. compute $D_{k_2}(C)$ for all keys k_2 ;
3. look for equal intermediate values;
4. confirm candidates with other message–ciphertext pairs.

The time becomes on the order of 2^k , not 2^{2k} , with large memory. The triple structure makes this direct attack harder, but it does not remove all generic reductions. Moreover, the block of only 64 bits imposes volume limits through the birthday effect. Therefore, “applying DES three times” does not solve the problem as well as adopting a modern cipher with a 128-bit block.

As an order of magnitude, 3DES with three keys is treated as offering about 112 bits of classical security, not the full 168 bits.

2.4 AES

AES is a substitution–permutation network, not a Feistel network. NIST ran an international public competition: it received fifteen candidates, selected five finalists after open analysis, and chose Rijndael in 2000. The choice considered security, software and hardware performance, flexibility, and implementation simplicity; it was not merely a comparison of key sizes.

All variants use a 128-bit block:

Variant	Key	Rounds
AES-128	128 bits	10
AES-192	192 bits	12
AES-256	256 bits	14

The state is a 4×4 matrix of bytes. The transformations are:

1. **SubBytes**: applies a bijective S-box to each byte;
2. **ShiftRows**: cyclically shifts the rows;
3. **MixColumns**: applies an invertible linear transformation to each column, over $GF(2^8)$;
4. **AddRoundKey**: XOR with the round subkey.

There is an initial AddRoundKey; the last round omits MixColumns.

2.4.1 Why can AES be decrypted?

- SubBytes is invertible because its S-box is a permutation;
- ShiftRows is inverted by shifts in the opposite direction;
- MixColumns uses an invertible matrix over $GF(2^8)$;
- AddRoundKey is its own inverse:

$$(x \oplus k) \oplus k = x;$$

- the key schedule makes it possible to reconstruct/generate the subkeys in the required order.

Decryption applies the inverses in reverse order. AES does not need to be Feistel because **each individual transformation** is invertible.

From the original key, the expansion algorithm generates $N_r + 1$ subkeys. Decryption starts with the last one and ends with the first. In the *Inverse Cipher*, the conceptual order is

$$\text{InvShiftRows}, \quad \text{InvSubBytes}, \quad \text{AddRoundKey}, \quad \text{InvMixColumns}.$$

In the *Equivalent Inverse Cipher* of FIPS 197, the round can be reorganized to have an appearance similar to encryption, applying InvMixColumns also to the internal subkeys. The two descriptions compute the same inverse permutation. The essential point is that the subkeys do not need to be “guessed”: they are derived deterministically from the key and used in reverse order.

2.4.2 Anatomy of an AES round

The 16 input bytes fill the state by columns:

$$S_{r,c} = \text{input}_{4c+r}, \quad 0 \leq r, c < 4.$$

This matters when doing a computation: the first four bytes form the first column, not the first row.

SubBytes. The S-box is not a randomly chosen table. For a nonzero byte, one computes the multiplicative inverse in $GF(2^8)$, using the modulus

$$x^8 + x^4 + x^3 + x + 1,$$

and then an affine transformation on the bits. Zero receives the conventional inverse zero before the affine transformation. For example, the S-box maps 53 to ED. The affine transformation avoids bad algebraic structures that would remain if only the inverse were used.

ShiftRows. Rows 0, 1, 2, 3 are shifted respectively by 0, 1, 2, 3 positions. Thus, bytes that were in the same column are spread across different columns before the next mixing.

MixColumns. Each column is multiplied by the matrix

$$\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}$$

over $GF(2^8)$. The classic example

$$\begin{pmatrix} D4 \\ BF \\ 5D \\ 30 \end{pmatrix} \mapsto \begin{pmatrix} 04 \\ 66 \\ 81 \\ E5 \end{pmatrix}$$

shows that each output byte depends on the four bytes of the column. Multiplication by 02 is a left shift followed by XOR with 1B when the most significant bit was 1; multiplying by 03 is multiplying by 02 and XORing with the original byte.

AddRoundKey. The state is XORed byte by byte with 128 bits of the key schedule. This is where the key enters; the other three operations are public.

SubBytes provides non-linearity; ShiftRows and MixColumns provide diffusion; AddRoundKey injects the secret. After several rounds, changing one bit of the input or of the key affects many bits of the output.

2.4.3 Key schedule and sizes that should not be confused

The AES block is always 128 bits. The key can be 128, 192, or 256 bits, and that choice changes the number of rounds. “AES-256” does not mean a 256-bit block. The key schedule expands the key into a 128-bit subkey for each AddRoundKey, using word rotation, application of the S-box, and round constants. It is not a simple repetition of the key.

2.5 Binary finite fields

A bitstring $b_{m-1} \dots b_1 b_0$ represents the polynomial

$$b_{m-1}x^{m-1} + \dots + b_1x + b_0$$

with coefficients in $GF(2)$. Addition and subtraction are XOR. In

$$GF(2^m) \cong GF(2)[x]/(f(x)),$$

$f(x)$ must be irreducible of degree m . One multiplies the polynomials and reduces the result modulo $f(x)$.

2.5.1 Example in $GF(2^4)$

With

$$f(x) = x^4 + x + 1,$$

we have

$$0101 = x^2 + 1, \quad 1101 = x^3 + x^2 + 1.$$

The product before reduction is

$$(x^2 + 1)(x^3 + x^2 + 1) = x^5 + x^4 + x^3 + 1.$$

Reducing modulo $x^4 + x + 1$:

$$x^5 + x^4 + x^3 + 1 \equiv x^3 + x^2,$$

therefore

$$0101 \cdot 1101 = 1100.$$

Also:

$$(1010)^{-1} = 1100,$$

since $(x^3 + x)(x^3 + x^2) \equiv 1 \pmod{x^4 + x + 1}$.

To check the inversion, multiply:

$$(x^3 + x)(x^3 + x^2) = x^6 + x^5 + x^4 + x^3.$$

From the modulus $x^4 + x + 1 = 0$ we get $x^4 \equiv x + 1$. Multiplying this relation by x and x^2 :

$$x^5 \equiv x^2 + x, \quad x^6 \equiv x^3 + x^2.$$

Hence

$$(x^3 + x^2) + (x^2 + x) + (x + 1) + x^3 = 1,$$

because repeated terms cancel in characteristic 2. The extended Euclidean algorithm could also find the inverse directly, replacing integers with polynomials over $GF(2)$.

2.5.2 Multiplication in GCM

GCM works in $GF(2^{128})$. A table of indices would have size on the order of 2^{128} and is infeasible. Multiplication is implemented through shift and XOR operations, with reduction by the polynomial fixed by the standard:

$$x^{128} + x^7 + x^2 + x + 1.$$

One way to perform the product is the *shift-and-XOR* algorithm. Initialize $Z = 0$ and $V = X$. Iterate over the 128 bits of Y ; when the current bit is 1, do $Z \leftarrow Z \oplus V$. Then shift V one position. If the bit that came out requires reduction, XOR with the representation of the reducing polynomial. In the bit convention used by GCM, this appears as

$$R = \text{E100} \dots 00$$

and the update is

$$V \leftarrow \begin{cases} V \gg 1, & \text{lsb}(V) = 0, \\ (V \gg 1) \oplus R, & \text{lsb}(V) = 1. \end{cases}$$

After 128 steps, $Z = X \cdot Y$. The cost is linear in 128, whereas a log/antilog table would have on the order of 2^{128} entries. It is important to fix the bit-order convention: shifting the other way is possible, but it changes the constant that represents the reduction.

2.6 Modes of operation

A block cipher encrypts a single fixed-length block. A **mode of operation** extends it to messages of arbitrary length, and the choice of mode determines which security properties hold (confidentiality only, or confidentiality plus authenticity) and what the requirements on the IV/nonce are.

Consider

$$m = m_1 \| m_2 \| \dots \| m_\ell.$$

2.6.1 ECB

$$c_i = E_k(m_i), \quad m_i = D_k(c_i).$$

Equal blocks produce equal ciphertexts. ECB is deterministic, reveals patterns, and is not CPA-secure. It should not be used for structured messages.

2.6.2 CBC

Choose $c_0 = IV$ uniform and unpredictable:

$$c_i = E_k(m_i \oplus c_{i-1}).$$

Decryption:

$$m_i = D_k(c_i) \oplus c_{i-1}.$$

The IV is transmitted with the ciphertext; it need not be secret. For the classical formulation of CPA security, it must be fresh and unpredictable.

2.6.2.1 Forward, backward, and padding For two blocks:

$$c_1 = E_k(m_1 \oplus IV), \quad c_2 = E_k(m_2 \oplus c_1).$$

Applying the inverse recovers

$$D_k(c_1) \oplus IV = (m_1 \oplus IV) \oplus IV = m_1,$$

$$D_k(c_2) \oplus c_1 = (m_2 \oplus c_1) \oplus c_1 = m_2.$$

CBC needs padding when the length is not a multiple of the block. The padding must be unambiguous and its validation must not reveal, through messages or timing, why a ciphertext was rejected; distinguishable responses can create a *padding oracle*. This reinforces a central point: CBC alone provides confidentiality, not authenticity.

If an attacker flips a bit of c_i , the plaintext block m_i becomes scrambled and the same bit of m_{i+1} is flipped in a controlled way, since

$$m_{i+1} = D_k(c_{i+1}) \oplus c_i.$$

This malleability exists even if the attacker does not know the key.

2.6.3 Chained CBC

Chained CBC uses the last ciphertext block of the previous message as the IV of the next. The value becomes known before the adversary chooses the next message. This enables queries constructed to test relations between blocks and breaks CPA. “Looking equivalent to concatenating messages” is not enough to guarantee security in the adaptive model.

2.6.3.1 Explicit attack on a predictable IV Suppose a query uses IV V to encrypt the block X and produces

$$C = E_k(X \oplus V).$$

In Chained CBC, C will be the public IV of the next message. In the challenge, choose

$$M_0 = X \oplus V \oplus C$$

and any $M_1 \neq M_0$. If M_0 is encrypted, the first block will be

$$C^* = E_k(M_0 \oplus C) = E_k(X \oplus V) = C.$$

Thus, answer $b' = 0$ when $C^* = C$ and $b' = 1$ otherwise. For $M_1 \neq M_0$, the bijectivity of E_k prevents equality in the other case. The adversary is always right. The CBC IV may be public after it is chosen; what this attack exploits is knowing it **before** choosing the message.

2.6.4 CTR

For nonce N and counter i :

$$z_i = E_k(N \parallel \langle i \rangle), \quad c_i = m_i \oplus z_i.$$

Decryption uses the same operation:

$$m_i = c_i \oplus z_i.$$

Advantages:

- parallelizable;
- random access to blocks;
- requires no padding;
- uses only the encryption direction of AES.

Critical rule: the key–nonce pair can never repeat. Repetition reuses the keystream.

If two messages use the same nonce,

$$c \oplus c' = m \oplus m'.$$

If m is known, the other message is recovered by

$$m' = c \oplus c' \oplus m.$$

It is also possible to flip a chosen bit of the plaintext by flipping the same bit of the ciphertext. CTR localizes errors, but it is malleable and must be combined with authentication.

2.6.5 GCM

GCM combines:

- CTR for confidentiality;
- GHASH to authenticate the ciphertext and associated data.

Define

$$H = E_k(0^{128})$$

and use multiplication in $GF(2^{128})$ to accumulate the AAD and ciphertext blocks. At a high level:

$$T = \text{MSB}_t(E_k(J_0) \oplus \text{GHASH}_H(A, C)).$$

The receiver recomputes T and accepts only if the comparison is valid. GCM is AEAD: it protects confidentiality and integrity, including associated data A that are not encrypted. Reusing a nonce in GCM is catastrophic: besides repeating the CTR keystream, it creates relations that compromise authentication.

2.6.6 Verification and structure of GHASH

Schematically, GHASH accumulates

$$A_1, \dots, A_u, C_1, \dots, C_v, \text{len}(A) \parallel \text{len}(C)$$

as a polynomial evaluation in H . The final block binds the lengths and avoids concatenation ambiguities. The associated data may be headers that must remain visible but must not be modified.

On reception:

1. derive $H = E_k(0^{128})$ and J_0 from the nonce;
2. recompute $S = \text{GHASH}_H(A, C)$;
3. compute $T' = \text{MSB}_t(E_k(J_0) \oplus S)$;
4. compare T' and the received tag without timing leakage;
5. accept only if they are equal; then recover the blocks via CTR, $m_i = c_i \oplus E_k(\text{inc}^i(J_0))$.

In a secure interface, unauthenticated plaintext is not delivered to the application. Changing the nonce, AAD, ciphertext, tag, or lengths must lead to rejection, except with the small forgery probability allowed by the tag size and the usage limits.

2.6.7 Error propagation and malleability

When a ciphertext bit is flipped, the effect depends on the mode. Without authentication, an attacker can exploit this to turn a ciphertext into another one that decrypts predictably.

Mode	Effect of a flipped bit in the ciphertext	Without authentication, can the attacker modify?
ECB	scrambles the corresponding block	yes
CBC	scrambles m_i and flips the corresponding bit of m_{i+1}	yes
CTR	flips exactly the same bit of m_i	yes
GCM	the tag must fail and the message must be rejected	forgery must be infeasible

2.7 Comparative summary of the modes

The table summarizes how the four modes differ in randomness, parallelizability, authentication, and their main weakness.

Mode	Randomness or nonce	Parallelizes encryption?	Authenticates?	Main problem
ECB	no	yes	no	reveals patterns
CBC	fresh IV	no	no	malleable; requires padding
CTR	unique nonce	yes	no	nonce reuse
GCM	unique nonce	yes	yes	nonce reuse

3 Part III — Modular Arithmetic, Groups and Primes

3.1 Divisibility, Greatest Common Divisor and Bézout

For integers a, b with $b \neq 0$, Euclidean division writes

$$a = qb + r, \quad 0 \leq r < |b|.$$

The Euclidean algorithm repeatedly uses

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

until the remainder is zero.

The **extended** Euclidean algorithm finds x, y such that

$$ax + by = \gcd(a, b).$$

During the Euclidean algorithm, each remainder is a linear combination of the two preceding numbers. By keeping the coefficients of those combinations, Bézout is obtained without having to redo the entire computation. An iterative form keeps

$$r_i = s_i a + t_i b.$$

Start with

$$(r_0, s_0, t_0) = (a, 1, 0), \quad (r_1, s_1, t_1) = (b, 0, 1).$$

For $q_i = \lfloor r_{i-1}/r_i \rfloor$, update

$$(r_{i+1}, s_{i+1}, t_{i+1}) = (r_{i-1}, s_{i-1}, t_{i-1}) - q_i(r_i, s_i, t_i).$$

The last nonzero remainder is the GCD, and the coefficients on the same row are the sought x, y .

When $\gcd(a, n) = 1$, the identity

$$ax + ny = 1$$

implies

$$ax \equiv 1 \pmod{n},$$

hence $x = a^{-1} \bmod n$.

3.1.1 Example: inverse of 17 modulo 101

$$101 = 5 \cdot 17 + 16, \quad 17 = 1 \cdot 16 + 1.$$

Going back:

$$1 = 17 - 16 = 17 - (101 - 5 \cdot 17) = 6 \cdot 17 - 101.$$

Therefore:

$$17^{-1} \equiv 6 \pmod{101}.$$

In the requested identity, the coefficients are explicitly

$$x = 6, \quad y = -1, \quad 17x + 101y = 1.$$

3.2 Modular Arithmetic

We write

$$a \equiv b \pmod{n}$$

when $n \mid (a - b)$. Congruences can be added and multiplied:

$$a \equiv b \pmod{n}, c \equiv d \pmod{n} \implies \begin{cases} a + c \equiv b + d \pmod{n}, \\ ac \equiv bd \pmod{n}. \end{cases}$$

Division is not a free operation. One may multiply by a^{-1} only when a is invertible modulo n , that is, when $\gcd(a, n) = 1$.

For example,

$$2x \equiv 2 \pmod{6}$$

does not allow one to conclude simply $x \equiv 1 \pmod{6}$: both $x = 1$ and $x = 4$ satisfy the congruence. Cancelling a non-invertible factor loses solutions or also requires adjusting the modulus.

3.3 Square-and-Multiply

To compute $a^e \bmod n$, write e in binary and do successive squarings, reducing modulo n at each step.

Example:

$$13 = (1101)_2 = 8 + 4 + 1.$$

For $3^{13} \bmod 17$:

$$\begin{aligned} 3^1 &\equiv 3, \\ 3^2 &\equiv 9, \\ 3^4 &\equiv 9^2 \equiv 13, \\ 3^8 &\equiv 13^2 \equiv 16 \pmod{17}. \end{aligned}$$

Thus:

$$3^{13} \equiv 3^8 \cdot 3^4 \cdot 3 \equiv 16 \cdot 13 \cdot 3 \equiv 12 \pmod{17}.$$

The number of multiplications is $O(\log e)$, not $O(e)$.

In the left-to-right algorithm, start with $r = 1$ and scan the bits of the exponent from the most significant to the least significant. For each bit, do

$$r \leftarrow r^2 \bmod n;$$

if the bit is 1, also do

$$r \leftarrow r \cdot a \bmod n.$$

This description makes clear why the cost grows with the number of bits of the exponent. In cryptographic implementations, branches and memory accesses that depend on secret bits must be avoided so as not to create side channels; this is an implementation matter, separate from the mathematical correctness of the algorithm.

3.4 Groups

A group $(G, \circ)$ satisfies:

1. closure: $a \circ b \in G$;
2. associativity;
3. existence of an identity e ;

4. existence of an inverse for each element.

If the operation is commutative, the group is abelian.

3.4.1 Additive and Multiplicative Groups

$$\mathbb{Z}_n^+ = \{0, 1, \dots, n-1\}$$

uses addition modulo n and has n elements.

$$\mathbb{Z}_n^* = \{a \in \{1, \dots, n-1\} : \gcd(a, n) = 1\}$$

uses multiplication modulo n and has

$$|\mathbb{Z}_n^*| = \varphi(n)$$

elements.

3.4.2 Group Order and Element Order

The order of the group is $|G|$. The order of $g \in G$ is the smallest positive integer r such that

$$g^r = e.$$

By Lagrange's theorem:

$$\text{ord}(g) \mid |G|.$$

In the additive group $\mathbb{Z}_n^+$:

$$\text{ord}(a) = \frac{n}{\gcd(a, n)}.$$

3.4.3 Cyclic Group and Generator

G is cyclic if there exists $g \in G$ such that

$$G = \langle g \rangle = \{g^0, g^1, \dots, g^{|G|-1}\}.$$

In that case, g is a generator and

$$\text{ord}(g) = |G|.$$

If the order of G is a prime q , every element other than the identity has order q and is a generator.

If the factorization of $|G| = N$ is known, there is an efficient test for a generator. Let $r_1, \dots, r_t$ be the **distinct prime** factors of N . Then

$$g \text{ generates } G \iff g^{N/r_i} \neq e \text{ for every } i.$$

If some test yields the identity, the order of g divides N/r_i and cannot be N . If none does, no prime factor can be removed from N , so the order is N .

3.5 Euler's Totient Function

$$\varphi(n) = |\mathbb{Z}_n^*|.$$

If

$$n = \prod_{i=1}^t p_i^{e_i},$$

then

$$\varphi(n) = n \prod_{i=1}^t \left(1 - \frac{1}{p_i}\right).$$

Important cases:

$$\varphi(p) = p - 1$$

for p prime, and

$$\varphi(pq) = (p - 1)(q - 1)$$

for distinct primes p, q .

Examples:

$$\varphi(100) = 40, \quad \varphi(101) = 100, \quad \varphi(1001) = 720,$$

since $1001 = 7 \cdot 11 \cdot 13$.

3.6 Fermat's and Euler's Theorems

If p is prime and $a \not\equiv 0 \pmod{p}$:

$$a^{p-1} \equiv 1 \pmod{p}.$$

More generally, if $\gcd(a, n) = 1$:

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

3.6.1 Use for Testing Compositeness

If

$$a^{n-1} \not\equiv 1 \pmod{n},$$

then n is composite.

The converse is false: some composite numbers pass for certain bases (pseudoprimes), and Carmichael numbers pass the Fermat test for every coprime base. Therefore, Fermat can prove compositeness when it finds a witness, but on its own it is not a good general primality test.

3.7 Chinese Remainder Theorem

If $\gcd(n_1, n_2) = 1$, the map

$$x \bmod (n_1 n_2) \mapsto (x \bmod n_1, x \bmod n_2)$$

is a bijection compatible with addition and multiplication:

$$\mathbb{Z}_{n_1 n_2} \cong \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2}.$$

For $n = pq$ with distinct primes:

$$\mathbb{Z}_n^* \cong \mathbb{Z}_p^* \times \mathbb{Z}_q^*.$$

This decomposition explains:

- the formula $\varphi(pq) = (p - 1)(q - 1)$;
- the complete proof of correctness of RSA;
- RSA decryption optimizations;
- why knowing p, q makes operations modulo n easier.

3.7.1 Explicit Reconstruction via CRT

For coprime moduli $n_1, \dots, n_t$, define

$$N = \prod_i n_i, \quad N_i = \frac{N}{n_i}, \quad u_i = N_i^{-1} \bmod n_i.$$

The solution of $x \equiv a_i \pmod{n_i}$ for every i is

$$x \equiv \sum_i a_i N_i u_i \pmod{N}.$$

Example:

$$x \equiv 2 \pmod{3}, \quad x \equiv 3 \pmod{5}.$$

We have $N = 15$, $N_1 = 5$, $5^{-1} \equiv 2 \pmod{3}$, $N_2 = 3$ and $3^{-1} \equiv 2 \pmod{5}$. Hence

$$x \equiv 2 \cdot 5 \cdot 2 + 3 \cdot 3 \cdot 2 = 38 \equiv 8 \pmod{15}.$$

Indeed, $8 \bmod 3 = 2$ and $8 \bmod 5 = 3$. Saying $\mathbb{Z}_{pq} \cong \mathbb{Z}_p \times \mathbb{Z}_q$ means more than there being the same number of elements: addition and multiplication modulo pq correspond to performing the operations separately in the two components. Restricting to the invertible elements gives $\mathbb{Z}_{pq}^* \cong \mathbb{Z}_p^* \times \mathbb{Z}_q^*$.

3.8 Prime Numbers and the Prime Number Theorem

If $\pi(x)$ counts the primes less than or equal to x , then

$$\pi(x) \sim \frac{x}{\ln x}.$$

Near x , the fraction of integers that are prime is approximately

$$\frac{1}{\ln x}.$$

Among odd candidates, the density is approximately

$$\frac{2}{\ln x}.$$

For 1024-bit numbers, $\ln(2^{1024}) = 1024 \ln 2 \approx 710$. Therefore, testing only odd numbers, one expects to find a prime after a few hundred candidates.

3.8.1 How Many Primes Have Exactly 100 Digits?

The desired number is

$$\pi(10^{100} - 1) - \pi(10^{99} - 1).$$

Using the approximation $\pi(x) \approx x / \ln x$,

$$\frac{10^{100}}{100 \ln 10} - \frac{10^{99}}{99 \ln 10} \approx 3.90 \times 10^{97}.$$

Using the bounds provided in the list,

$$\ln x - \frac{3}{2} \leq \frac{x}{\pi(x)} \leq \ln x - \frac{1}{2},$$

and applying the lower bound at 10^{100} and the upper bound at 10^{99} (and vice versa), one obtains approximately

$$3.91 \times 10^{97} \lesssim \#\{100\text{-digit primes}\} \lesssim 3.93 \times 10^{97}.$$

Replacing $10^d - 1$ by 10^d does not affect the digits shown at this scale.

3.9 Miller–Rabin

To test an odd integer $n > 2$, write

$$n - 1 = 2^s d, \quad d \text{ odd}.$$

Choose a random base $a \in \{2, \dots, n - 2\}$ and compute

$$x = a^d \bmod n.$$

One round is:

1. if $x = 1$ or $x = n - 1$, the base did not detect compositeness;

2. repeat at most $s - 1$ times:

$$x \leftarrow x^2 \bmod n;$$

if $x = n - 1$, the base did not detect compositeness;

3. if no value was $n - 1$, declare n composite.

If n is prime, all valid bases pass. If n is odd and composite, at most $1/4$ of the bases are *strong liars*. With t independent rounds:

$$\Pr[\text{composite accepted as prime}] \leq 4^{-t}.$$

The test is probabilistic in the sense of being able to accept a composite; when it declares “composite”, the conclusion is certain.

3.9.1 Two Rounds Worked by Hand

Composite $n = 21$, **base** $a = 2$. Write

$$20 = 2^2 \cdot 5, \quad s = 2, d = 5.$$

Compute

$$x = 2^5 \bmod 21 = 11.$$

It is neither 1 nor 20. There is only one more squaring:

$$x^2 \equiv 11^2 \equiv 16 \pmod{21},$$

which is also not 20. The base 2 witnesses that 21 is composite.

Prime $n = 29$, **base** $a = 2$. Now

$$28 = 2^2 \cdot 7, \quad 2^7 \equiv 12 \pmod{29}.$$

On the first squaring,

$$12^2 \equiv 144 \equiv 28 \equiv -1 \pmod{29},$$

so the round passes, as must happen for every valid base when n is prime.

3.9.2 Two Meanings of “Error Probability”

For a **fixed composite** n , a random Miller–Rabin base lets the number pass with probability at most $1/4$. After t independent bases, the bound is 4^{-t} .

The question “what is the chance that the number returned by the generator is composite, given that it passed?” is instead conditional and depends on the density of primes among the candidates. Table 4.3 of the HAC provides tighter bounds for random search. A 100-digit number has about 333 bits; the table does not have exactly that row. For one round, it lists 2^{-19} at 300 bits and 2^{-28} at 350 bits, indicating the scale of the applicable bound. One must not confuse this posterior probability with the universal bound of $1/4$ for a base applied to a previously fixed composite.

3.9.3 Generating a Large Prime

1. generate a uniform candidate with the most significant bit equal to 1;
2. force the least significant bit to 1, making it odd;
3. eliminate divisibility by small primes;
4. run several rounds of Miller–Rabin;
5. if it fails, generate another candidate.

A direct implementation of one round follows this logic:

```
milller_rabin_round(n, a):
    if gcd(a,n) > 1: return "composite"
    write n-1 = 2^s d, with d odd
    x = a^d mod n          # use square-and-multiply
    if x = 1 or x = n-1: return "passed"
    repeat s-1 times:
        x = x^2 mod n
        if x = n-1: return "passed"
    return "composite"
```

The test with t rounds draws new bases and returns “probable prime” only if all of them pass. Cases $n < 4$, even numbers and bases outside the range must be handled in the function’s interface.

To generate a 1024-bit prime, step 1 must draw exactly in that range:

$$2^{1023} \leq n < 2^{1024}.$$

Fixing the most significant bit guarantees 1024 bits; fixing the least significant bit eliminates even candidates. Each approved candidate is a **probable prime**, unless a test that produces a deterministic proof of primality is subsequently used.

3.10 Orders: Examples from the List

These are the concrete order computations requested in the problem set. They show how to compute orders and identify generators in both additive and multiplicative notation.

3.10.1 Additive Group $\mathbb{Z}_{36}^+$

Because the additive group $\mathbb{Z}_{36}^+$ is cyclic, the order of an element is easy to compute: it is $36/\gcd(a, 36)$. The elements are grouped by order below.

Using

$$\text{ord}(a) = \frac{36}{\gcd(a, 36)},$$

the elements are grouped by order:

Order	Elements
1	0
2	18
3	12, 24
4	9, 27
6	6, 30
9	4, 8, 16, 20, 28, 32
12	3, 15, 21, 33
18	2, 10, 14, 22, 26, 34
36	1, 5, 7, 11, 13, 17, 19, 23, 25, 29, 31, 35

3.10.2 Multiplicative Group $\mathbb{Z}_{36}^*$

$$\mathbb{Z}_{36}^* = \{1, 5, 7, 11, 13, 17, 19, 23, 25, 29, 31, 35\}, \quad \varphi(36) = 12.$$

Order	Elements
1	1
2	17, 19, 35
3	13, 25
6	5, 7, 11, 23, 29, 31

No element has order 12, so $\mathbb{Z}_{36}^*$ is not cyclic.

3.10.3 Group $\mathbb{Z}_{37}^*$

Since 37 is prime:

$$|\mathbb{Z}_{37}^*| = 36.$$

It is cyclic. Its generators are the elements of order 36:

$$2, 5, 13, 15, 17, 18, 19, 20, 22, 24, 32, 35.$$

There are exactly

$$\varphi(36) = 12$$

generators, as expected.

The complete table asked for in the list is:

Order	Elements of $\mathbb{Z}_{37}^*$
1	1
2	36
3	10, 26
4	6, 31
6	11, 27
9	7, 9, 12, 16, 33, 34
12	8, 14, 23, 29
18	3, 4, 21, 25, 28, 30
36	2, 5, 13, 15, 17, 18, 19, 20, 22, 24, 32, 35

The counts in each row are consistent with the cyclic structure: for each divisor d of 36, there are $\varphi(d)$ elements of order d , and the sum of these counts is 36.

3.10.4 Table of Powers Modulo 17

3 has order 16 in $\mathbb{Z}_{17}^*$. The sequence is:

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$3^i \bmod 17$	1	3	9	10	13	5	15	11	16	14	8	7	4	12	2	6	1

If $a = 3^j$, then

$$a^{-1} = 3^{16-j} \pmod{17}.$$

A table of indices turns inversion into a lookup and a subtraction of exponents. For large cryptographic groups, building the entire table would cost memory proportional to the order of the group. # Part IV — Public-Key Cryptography

3.11 Overview

In public-key cryptography, each user has:

$$(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda).$$

- pk is public;
- sk remains secret;
- anyone can encrypt to the owner of pk;
- only whoever holds sk should be able to decrypt.

This does not eliminate the need to authenticate public keys. Without certificates or another authentication mechanism, an adversary can substitute a key and carry out a man-in-the-middle attack.

3.12 RSA

RSA is a public-key scheme built on the assumed difficulty of factoring. The public key is (n, e) and the secret key is d ; encryption raises the message to e modulo n , and decryption raises the ciphertext to d .

3.12.1 Key generation

1. choose large distinct primes p, q ;
2. compute

$$n = pq;$$

3. compute

$$\varphi(n) = (p-1)(q-1);$$

4. choose e with

$$\gcd(e, \varphi(n)) = 1;$$

5. compute

$$d \equiv e^{-1} \pmod{\varphi(n)}.$$

Keys:

$$\text{pk} = (n, e), \quad \text{sk} = (n, d)$$

and, in practice, one also stores p, q and CRT parameters to speed up decryption.

3.12.2 Small example of generation, encryption, and decryption

The following numbers are didactic, not secure. Choose

$$p = 61, \quad q = 53, \quad n = 3233, \quad \varphi(n) = 60 \cdot 52 = 3120.$$

Take $e = 17$. Since $\gcd(17, 3120) = 1$, the inverse exists. The extended Euclidean algorithm gives

$$d = 17^{-1} \bmod 3120 = 2753,$$

since $17 \cdot 2753 = 1 + 15 \cdot 3120$. For $m = 65$:

$$c = 65^{17} \bmod 3233 = 2790,$$

$$\widehat{m} = 2790^{2753} \bmod 3233 = 65.$$

The powers are computed with Square-and-Multiply. This example brings together the complete sequence: choose primes, compute φ , check the GCD, obtain the inverse, exponentiate to encrypt, and exponentiate to decrypt.

3.12.3 Textbook encryption and decryption

For $m \in \mathbb{Z}_n$, the textbook RSA map is deterministic: encryption raises to e , decryption raises to d .

$$c = m^e \bmod n, \quad \widehat{m} = c^d \bmod n.$$

3.12.4 Correctness when $\gcd(m, n) = 1$

Since

$$ed \equiv 1 \pmod{\varphi(n)},$$

there exists t such that

$$ed = 1 + t\varphi(n).$$

By Euler's theorem:

$$m^{\varphi(n)} \equiv 1 \pmod{n}.$$

Therefore:

$$c^d \equiv m^{ed} = m^{1+t\varphi(n)} \equiv m \left(m^{\varphi(n)}\right)^t \equiv m \pmod{n}.$$

3.12.5 Correctness for every $m \in \mathbb{Z}_n$

The condition $\gcd(m, n) = 1$ is necessary to apply Euler directly, but RSA remains correct for non-invertible messages. Prove it separately modulo p and modulo q .

Modulo p :

- if $m \equiv 0 \pmod{p}$, then $m^{ed} \equiv m \equiv 0 \pmod{p}$;
- otherwise, Fermat gives $m^{p-1} \equiv 1 \pmod{p}$, and $ed \equiv 1 \pmod{p-1}$ implies $m^{ed} \equiv m \pmod{p}$.

The same holds modulo q . By CRT:

$$m^{ed} \equiv m \pmod{pq}.$$

3.12.6 CRT-accelerated decryption

Instead of one exponentiation modulo n , compute

$$m_p = c^{d_p} \bmod p, \quad d_p = d \bmod (p-1),$$

$$m_q = c^{d_q} \bmod q, \quad d_q = d \bmod (q-1),$$

and reconstruct the unique $m \bmod n$ with these two residues. The operations modulo p and q use numbers with approximately half the bits of n and are much faster. Implementations also store, for example, $q^{-1} \bmod p$ to perform the recombination.

3.12.7 Why $e = 65537$?

$$65537 = 2^{16} + 1$$

has a binary representation with only two bits 1, making public exponentiation efficient by Square-and-Multiply. It is large enough to avoid several problems associated with extremely small public exponents and is prime, making it easy to obtain

$$\gcd(e, \varphi(n)) = 1.$$

$e = 3$ is not automatically insecure when RSA is implemented with correct padding. The problem is using it with textbook RSA:

- if $m^3 < n$, then $c = m^3$ as an integer and it suffices to extract the cube root;
- sending the same message to three coprime moduli lets one reconstruct m^3 via CRT (broadcast attack);
- algebraic structure and incorrect padding can give rise to other attacks.

3.12.8 Factoring versus inverting RSA

Knowing p, q lets one compute $\varphi(n)$, obtain d , and break RSA. Therefore:

$$\text{factor } n \implies \text{invert RSA.}$$

The converse is not known in general: there is no proof that any algorithm capable of inverting RSA instances necessarily factors n . “RSA is as hard as factoring” should not be asserted as a demonstrated equivalence.

3.12.9 Knowing $\varphi(n)$ implies factoring $n = pq$

Since

$$\varphi(n) = (p-1)(q-1) = pq - p - q + 1,$$

we have

$$p + q = n - \varphi(n) + 1.$$

Therefore p, q are roots of

$$x^2 - (n - \varphi(n) + 1)x + n = 0.$$

For

$$n = 4294049777, \quad \varphi(n) = 4293918720,$$

we obtain

$$p + q = 131058$$

and discriminant

$$\Delta = 131058^2 - 4 \cdot 4294049777 = 256 = 16^2.$$

Therefore:

$$p = \frac{131058 + 16}{2} = 65537, \quad q = \frac{131058 - 16}{2} = 65521.$$

3.12.10 Message not coprime with n

If a random message satisfies

$$\gcd(m, n) \neq 1,$$

then computing $\gcd(m, n)$ itself reveals p or q . This is the serious problem; it is not a failure of RSA’s correctness.

The probability for m uniform modulo $n = pq$ is:

$$\Pr[\gcd(m, n) \neq 1] = 1 - \frac{\varphi(n)}{n} = \frac{1}{p} + \frac{1}{q} - \frac{1}{pq}.$$

For large RSA primes it is negligible, but if it occurs the factorization becomes immediate.

3.12.11 Procedure for the exercise with 12-digit primes

1. choose two distinct primes p, q with $10^{11} \leq p, q < 10^{12}$;
2. compute $n = pq$ and confirm that it has approximately 24 digits;
3. compute $\varphi(n) = (p-1)(q-1)$;
4. try $e = 65537$ and confirm $\gcd(e, \varphi(n)) = 1$; if it fails, change the primes or choose another valid exponent;
5. use the extended Euclidean algorithm to obtain d and verify $ed \bmod \varphi(n) = 1$;
6. choose $0 \leq m < n$, compute $c = m^e \bmod n$, and confirm $c^d \bmod n = m$.

12-digit primes are adequate only for the exercise. The final test should include the algebraic checks; printing only four numbers does not show that the key was generated correctly.

3.13 Why is textbook RSA not secure?

The raw RSA map $m \mapsto m^e \bmod n$ is a deterministic trapdoor permutation. Used directly as an encryption scheme it fails the standard security notions for two structural reasons: it is deterministic and multiplicatively malleable.

3.13.1 Determinism

$$\text{Enc}_{\text{pk}}(m) = m^e \bmod n$$

always produces the same ciphertext. In the CPA game, the adversary encrypts m_0 publicly and compares it with the challenge.

3.13.2 Multiplicative malleability

$$\text{Enc}(m_1)\text{Enc}(m_2) \equiv (m_1 m_2)^e \equiv \text{Enc}(m_1 m_2) \pmod{n}.$$

An adversary alters a ciphertext predictably without knowing the message.

3.13.3 Small messages and structure

Textbook RSA preserves algebraic relations and introduces no randomness. Real RSA needs padding/encoding with a security analysis, such as OAEP for encryption.

3.14 OAEP

OAEP applies a Feistel-like transformation, with mask functions G and H , before the RSA operation.

In simplified form, for an encoded message M and random seed r :

$$X = M \oplus G(r), \quad Y = r \oplus H(X).$$

The block $X\|Y$ is then processed by RSA. To reverse:

$$r = Y \oplus H(X), \quad M = X \oplus G(r).$$

Objectives:

- make encryption probabilistic;
- make the entire message depend on all bits of X and Y ;
- prevent direct message testing;
- prevent the simple use of the multiplicative structure of textbook RSA.

The *all-or-nothing* property means that losing or altering part of the block prevents correctly recovering either the seed or the message. In real implementations, OAEP includes a label, a hash of the label, and precise encoding rules; it is not enough to invent a similar padding.

3.15 Diffie–Hellman

Public parameters:

$$G = \langle g \rangle, \quad |G| = q.$$

Flow:

1. Alice chooses

$$a \xleftarrow{\$} \mathbb{Z}_q$$

- and sends $A = g^a$;
 2. Bob chooses

$$b \xleftarrow{\$} \mathbb{Z}_q$$

- and sends $B = g^b$;
 3. Alice computes

$$K_A = B^a = g^{ab};$$

4. Bob computes

$$K_B = A^b = g^{ab}.$$

A KDF must transform the group secret into separate symmetric keys.

3.15.1 Complete example and verification

Use the prime $p = 23$ and $g = 5$. One verifies that 5 has order 22 modulo 23, so it generates all of $\mathbb{Z}_{23}^*$. Alice chooses $a = 6$ and Bob chooses $b = 15$.

$$A = g^a = 5^6 \bmod 23 = 8, \quad B = g^b = 5^{15} \bmod 23 = 19.$$

Alice receives B and computes

$$K_A = B^a = 19^6 \bmod 23 = 2.$$

Bob receives A and computes

$$K_B = A^b = 8^{15} \bmod 23 = 2.$$

Both arrive at the same value because

$$K_A = (g^b)^a = g^{ab} = g^{ba} = (g^a)^b = K_B.$$

An observer who sees p, g, A, B would need to compute g^{ab} without knowing a or b ; this is exactly the CDH assumption. The exponentiations come out by Square-and-Multiply, never by repeated multiplication.

In real use, g^{ab} is processed by a KDF before becoming a session key. It is a group element, not a 128-bit key: using its bits directly wastes entropy and can inherit the algebraic structure of the group.

3.15.2 Why “establishment”, not “exchange”?

Neither party chooses the final value

$$g^{ab}$$

in advance. Each contributes an exponent, and the key arises from the combination. The key is not simply transported from one party to the other.

3.15.3 Limitation: lack of authentication

Plain DH resists a passive observer under suitable assumptions, but not an active adversary. Mallory can establish one key with Alice and another with Bob. To prevent MITM, authenticate the ephemeral values with signatures, certificates, or a previously shared key.

3.16 The DLP, CDH, and DDH problems

In $G = \langle g \rangle$:

- **DLP:** given g and $h = g^x$, find x ;
- **CDH:** given g^a, g^b , compute g^{ab} ;
- **DDH:** distinguish

$$(g^a, g^b, g^{ab})$$

from

$$(g^a, g^b, g^c)$$

for random c .

In general:

resolver DLP $\implies$ resolver CDH $\implies$ resolver DDH,

but the converses are not known in all groups.

3.17 ElGamal

ElGamal is a public-key encryption scheme in a cyclic group, derived from Diffie–Hellman: the receiver’s fixed contribution $h = g^x$ masks the message with an ephemeral shared secret h^y .

3.17.1 Generation

In $G = \langle g \rangle$ of order q :

$$x \xleftarrow{\$} \mathbb{Z}_q, \quad h = g^x.$$

Keys:

$$\text{pk} = (G, q, g, h), \quad \text{sk} = x.$$

3.17.2 Encryption

For $m \in G$, choose a new

$$y \xleftarrow{\$} \mathbb{Z}_q$$

and compute

$$c_1 = g^y, \quad c_2 = m \cdot h^y.$$

The ciphertext is

$$c = (c_1, c_2).$$

3.17.3 Decryption

$$m = \frac{c_2}{c_1^x} = \frac{m \cdot (g^x)^y}{(g^y)^x} = m.$$

ElGamal is the non-interactive perspective of Diffie–Hellman: $h = g^x$ works as the receiver’s fixed contribution, g^y is the sender’s ephemeral contribution, and $h^y = g^{xy}$ is the shared secret used as a mask.

With a fresh exponent y for each encryption and the DDH assumption, ElGamal is CPA-secure. Reusing y relates the ciphertexts:

$$\frac{c_2}{c_2'} = \frac{m}{m'}.$$

3.17.4 Complete example

With the same parameters $p = 23$ and $g = 5$, choose the secret key $x = 6$, so

$$h = g^x = 5^6 \bmod 23 = 8.$$

To encrypt $m = 7$, the sender chooses the ephemeral exponent $y = 3$:

$$c_1 = g^y = 5^3 \bmod 23 = 10,$$

$$c_2 = m \cdot h^y = 7 \cdot 8^3 \bmod 23 = 7 \cdot 6 \bmod 23 = 19.$$

The ciphertext is $(c_1, c_2) = (10, 19)$. The receiver computes

$$c_1^x = 10^6 \bmod 23 = (5^3)^6 = 5^{18} \bmod 23 \equiv 6,$$

and recovers

$$m = c_2 \cdot (c_1^x)^{-1} = 19 \cdot 6^{-1} \bmod 23 = 19 \cdot 4 \bmod 23 = 7,$$

since $6^{-1} \equiv 4 \pmod{23}$. The value $y = 3$ must be fresh for each message: repeating it would produce

$$\frac{c_2}{c'_2} = \frac{m}{m'},$$

which reveals the ratio between the two plaintexts with no computational effort at all.

3.18 Group choice and safe primes

If

$$p = 2q + 1$$

with p, q prime, then p is a *safe prime*. The set of quadratic residues modulo p forms a subgroup of order q .

For $b \in \mathbb{Z}_p^*$, define

$$c = b^2 \pmod{p}.$$

Then c belongs to the subgroup of quadratic residues. Since this subgroup has prime order q :

- if $c = 1$ (cases $b = \pm 1$), its order is 1;
- otherwise, its order is q and c is a generator of the subgroup.

Using the prime-order subgroup facilitates the analysis and avoids small-order elements.

3.18.1 b or c : which to use?

The element b generates a group of order $2q$ that contains -1 , of order 2. If the implementation accepts invalid inputs, an active adversary can force the secret into this small subgroup and recover information from the exponent's residue. Meanwhile $c = b^2$ belongs to the subgroup of quadratic residues, of prime order q , whose only possible orders are 1 and q .

That is why c is the cleaner choice as a parameter: it eliminates small subgroups and makes security depend on a single prime parameter q , which is also the exponent of the best generic attack, $O(\sqrt{q})$.

3.19 Factoring algorithms

These algorithms recover a nontrivial factor of a composite n . They are the reason RSA moduli must be large, and they illustrate the distinction between generic and special-purpose attacks.

3.19.1 Pollard–rho for factoring

Choose an iterated function, for example

$$f(x) = x^2 + c \pmod{n}.$$

Generate two sequences with Floyd:

$$x \leftarrow f(x), \quad y \leftarrow f(f(y)).$$

At each step compute

$$d = \gcd(|x - y|, n).$$

- $d = 1$: continue;
- $1 < d < n$: d is a factor;
- $d = n$: the attempt degenerated; change the seed or c .

Intuition: modulo a prime factor p , the sequence enters a cycle after about $O(\sqrt{p})$ steps by the birthday phenomenon. A collision modulo p makes $p \mid (x - y)$, revealing it through the gcd.

3.19.2 Worked example

Factor

$$n = 8051.$$

Use $f(x) = x^2 + 1 \pmod{n}$, seed $x_0 = 2$, and the Floyd pair $(x \leftarrow f(x), y \leftarrow f(f(y)))$:

step	x	y	$\gcd(x - y , n)$
1	5	26	1

step	x	y	$\gcd(x - y , n)$
2	26	7474	1
3	677	871	97

At the third step 97 divides n , so the algorithm stops and returns

$$8051 = 97 \cdot 83.$$

The computation is short because n is small; what matters is the growth rate. The expected number of steps is $O(\sqrt{p})$, where p is the smallest prime factor, and not $O(\sqrt{n})$: the collision the algorithm looks for occurs modulo p , and it is the birthday bound applied to the group generated by p that determines the cost.

3.20 Discrete logarithm algorithms

Given g and $h = g^x$ in a group of order q , the discrete log problem asks for x . Generic algorithms run in roughly $\sqrt{q}$ group operations and set the benchmark for choosing group sizes.

3.20.1 Exhaustive search

Test

$$g^0, g^1, \dots$$

until finding h . Cost $O(q)$ for a group of order q .

3.20.2 Baby-Step/Giant-Step

Uses a decomposition of the exponent and a table of approximately $\sqrt{q}$ elements:

$$\text{time } O(\sqrt{q}), \quad \text{memory } O(\sqrt{q}).$$

3.20.3 Pollard–rho for DLP

Keep states with a known representation

$$X_i = g^{a_i} h^{b_i}.$$

An iteration function updates X_i, a_i, b_i . When a collision is found

$$X_i = X_j,$$

we have, writing $h = g^x$:

$$g^{a_i + b_i x} = g^{a_j + b_j x}.$$

Therefore:

$$(b_i - b_j)x \equiv a_j - a_i \pmod{q}.$$

If $b_i - b_j$ is invertible modulo q :

$$x \equiv (a_j - a_i)(b_i - b_j)^{-1} \pmod{q}.$$

Expected complexity:

$$O(\sqrt{q})$$

group operations and constant memory. A degenerate collision requires restarting.

3.20.4 Index Calculus

Index Calculus exploits the specific representation of the elements of $\mathbb{Z}_p^*$ as integers that can factor over a base of small factors.

1. choose a base

$$\mathcal{B} = \{p_1, \dots, p_t\};$$

2. look for exponents r for which

$$g^r \bmod p = \prod_i p_i^{e_i}$$

is *smooth*;

3. obtain linear equations

$$r \equiv \sum_i e_i \log_g(p_i) \pmod{p-1};$$

4. solve the logs of the base elements;

5. combine h with a power of g until obtaining another smooth value and recover $\log_g h$.

It is subexponential in $\mathbb{Z}_p^*$ and beats generic attacks. It does not work directly in elliptic-curve groups because curve points do not have an analogous smooth factorization. That is why, for well-chosen ECC parameters, the best generic attacks remain on the order of $\sqrt{q}$.

3.21 Elliptic curves

Over $\mathbb{F}_p$, a short Weierstrass curve is:

$$E : y^2 = x^3 + ax + b \pmod{p},$$

with the non-singularity condition

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p}.$$

The curve's points, plus the point at infinity $\mathcal{O}$, form an abelian group under geometric/algebraic addition.

3.21.1 Point addition

For $P = (x_1, y_1)$ and $Q = (x_2, y_2)$, $P \neq \pm Q$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}.$$

For doubling $P = Q$:

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}.$$

This requires $2y_1$ to be invertible modulo p . Since p is prime, the only bad case is $y_1 = 0$; then the tangent is vertical and $2P = \mathcal{O}$ (the point has order 2). Otherwise $y_1 \neq 0$ and the division is valid.

Then:

$$x_3 = \lambda^2 - x_1 - x_2 \pmod{p},$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}.$$

The divisions mean multiplication by the modular inverse.

3.21.2 Worked example

Consider

$$E : y^2 = x^3 + 2x + 2 \pmod{17}.$$

Since

$$4a^3 + 27b^2 = 32 + 108 = 140 \equiv 4 \pmod{17} \neq 0,$$

the curve is non-singular. It has 18 affine points plus the point at infinity, totaling

$$|E| = 19,$$

a prime number. Take

$$P = (5, 1),$$

which is on the curve, since $5^3 + 2 \cdot 5 + 2 = 137 \equiv 1 \pmod{17}$. Here $P = Q$, so we use the doubling formula:

$$\lambda = \frac{3 \cdot 5^2 + 2}{2 \cdot 1} = \frac{77}{2} \equiv 9 \cdot 2^{-1} \equiv 9 \cdot 9 \equiv 13 \pmod{17},$$

since $2^{-1} \equiv 9 \pmod{17}$. It follows that

$$x_3 = \lambda^2 - 2x_1 = 169 - 10 = 159 \equiv 6 \pmod{17},$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 13(5 - 6) - 1 = -14 \equiv 3 \pmod{17}.$$

Therefore

$$2P = (6, 3).$$

Continuing, $3P = (10, 6)$. Since $|E| = 19$ is prime, every nonzero point has order 19; in particular the multiples

$$P, 2P, 3P, \dots, 18P$$

are all distinct and

$$19P = \mathcal{O}.$$

It is this large, prime order that makes the group useful: the ECDLP with base P costs about $\sqrt{19}$ steps in this example and $\sqrt{q}$ in the real case.

3.21.3 ECDLP

Given a base point P of large order and

$$Q = xP,$$

the ECDLP asks for the scalar x . Calling it “discrete logarithm” is strange only because the group notation is additive: a power g^x does not appear, but the scalar multiplication xP is the analogous operation.

3.22 Work factors and key sizes

Problem/group	Best relevant classical family	Consequence
factoring/RSA	subexponential algorithms	moduli need to be large
DLP in $\mathbb{Z}_p^*$	subexponential Index Calculus	p needs to be large
well-parameterized ECDLP	Pollard-rho $O(\sqrt{q})$	ECC keys are smaller

Rough classical strength comparison:

Strength	RSA	DH/DSA in finite field	ECC	Symmetric
112 bits	2048	$L = 2048, N = 224$	224–255	at least 112
128 bits	3072	$L = 3072, N = 256$	256–383	AES-128
192 bits	7680	$L = 7680, N = 384$	384–511	AES-192
256 bits	15360	$L = 15360, N = 512$	512+	AES-256

L is the size of the modulus and N the order of the subgroup. The table compares **classical** security; a large-scale quantum computer would change the comparison for RSA, DH, and ECC. # Part V — Hashes, MACs and Signatures

3.23 Different Objectives

Primitive	Uses a secret?	Who verifies?	Goal
CRC	no	anyone	detect accidental errors
cryptographic hash	no	anyone	digest with properties against attacks
MAC	yes, shared key	whoever knows the key	symmetric integrity and authentication
digital signature	private key to sign	anyone with the public key	integrity, origin and public verifiability

The **CRC** (*cyclic redundancy check*) is the weakest level in the table. The message is viewed as a polynomial $M(x)$ over $GF(2)$, and the transmitted value is the remainder

$$R(x) = M(x) \bmod G(x)$$

for a fixed, public generator polynomial $G(x)$. It is an error-detection code, not a security mechanism. In particular it is **linear**: changing M in a controlled way changes R predictably, so an adversary recomputes the check without knowing anything secret.

A hash does not authenticate by itself: anyone who changes m can also compute $H(m')$. Authentication requires a key (MAC) or a signature.

3.24 Hash Function

A hash function maps arbitrary inputs to a fixed-size output:

$$H : \{0,1\}^* \longrightarrow \{0,1\}^n.$$

3.24.1 Preimage resistance / one-wayness

Given y , it must be hard to find x such that

$$H(x) = y.$$

Generic attack: approximately 2^n evaluations.

3.24.2 Second-preimage resistance / weak collision resistance

Given x , it must be hard to find $x' \neq x$ such that

$$H(x') = H(x).$$

Generic attack: approximately 2^n evaluations.

3.24.3 Collision resistance / strong collision resistance

It must be hard to find **any** pair $x \neq x'$ such that

$$H(x) = H(x').$$

Generic attack: approximately $2^{n/2}$ evaluations, by the birthday paradox.

Collision resistance implies, in usual contexts, a stronger guarantee than allowing the adversary to receive a fixed first message. Do not confuse the two: in a collision, the attacker chooses both messages; in a second preimage, one of them has already been fixed.

3.25 Birthday Paradox

Choose q uniform and independent values from a set of size N . The exact probability of no collision is:

$$\Pr[\text{no collision}] = \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right).$$

Therefore:

$$\Pr[\text{collision}] = 1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right).$$

For $q \ll N$:

$$\Pr[\text{collision}] \approx 1 - \exp\left(-\frac{q(q-1)}{2N}\right).$$

For probability 1/2:

$$q \approx \sqrt{2N \ln 2} \approx 1.177\sqrt{N}.$$

3.25.1 Example with 36500 possibilities

$$\sqrt{2 \cdot 36500 \ln 2} \approx 224.94.$$

By the exact product:

$$\Pr[\text{collision with 225}] \approx 0.49934,$$

$$\Pr[\text{collision with 226}] \approx 0.50243.$$

Therefore, the smallest q with probability at least 1/2 is

$$q = 226.$$

3.25.2 The wallets example

With $P = 10^{10}$ people, $A = 10^7$ addresses per person and space $N = 2^{160}$, the number of pairs of addresses belonging to different people is approximately

$$\binom{P}{2} A^2.$$

Since the probability is very small:

$$\Pr[\text{some coincidence between people}] \approx \frac{\binom{P}{2} A^2}{2^{160}} \approx 3.42 \times 10^{-15}.$$

The total number of addresses is enormous, but 2^{160} is much larger.

The same phenomenon governs the discrete-logarithm and factoring attacks. Pollard–rho searches for a collision inside the group generated by the factor p (factoring) or inside the group where the discrete log is sought (DLP). Hence its costs are $O(\sqrt{p})$ and $O(\sqrt{q})$: the birthday scale applied to the size of the **effective** subgroup, and not to the size of n nor to that of p as a whole.

3.26 Collision Attack against a Hash

For an n -bit hash:

1. generate distinct messages;
2. compute their hashes;
3. store and look for two equal outputs;
4. after about $2^{n/2}$ attempts, a collision becomes likely.

Consequence: an n -bit hash provides at most about $n/2$ bits of security against collisions. To obtain approximately 128 bits against collision, one wants an output of at least 256 bits.

This observation definitively separated three notions in the security definition of hashes. “Hard to invert” does not automatically imply “hard to find a collision”.

3.27 Known Hashes

- **MD5**: practical collisions; do not use for security;
- **SHA-1**: collision resistance broken and being retired;
- **SHA-2**: includes SHA-256 and SHA-512; recommended family;
- **SHA-3**: construction based on Keccak; standardized alternative to the SHA-2 family.

The classification depends on the property and the application. An algorithm broken for collisions must not be used in signatures, even if preimage attacks are still far more expensive.

3.28 MAC

A MAC is a triple:

$$(\text{Gen}, \text{Mac}, \text{Vrfy}).$$

$$k \leftarrow \text{Gen}(1^\lambda), \quad t \leftarrow \text{Mac}_k(m),$$

$$\text{Vrfy}_k(m, t) \in \{0, 1\}.$$

Correctness:

$$\Pr[\text{Vrfy}_k(m, \text{Mac}_k(m)) = 1] = 1.$$

Usual security notion: even after querying tags of chosen messages, an adversary must not produce a valid tag for a new message. This is a form of unforgeability under chosen-message attack.

3.28.1 Dangerous Constructions

One must not assume that

$$H(k\|m)$$

is a secure MAC for any hash. In Merkle–Damgård constructions, there can be a length-extension attack. Other naive concatenations also lack proof and may have ambiguities.

3.28.2 HMAC

HMAC uses two applications of the hash with domain separation:

$$\text{HMAC}_K(m) = H((K' \oplus \text{opad})\|H((K' \oplus \text{ipad})\|m)).$$

K' is the key adjusted to the block size of the hash. The values `ipad` and `opad` separate the inner domain from the outer one.

3.28.3 CBC-MAC

Basic CBC-MAC processes blocks as CBC with a fixed IV and uses the last block as the tag. The basic construction is secure only for fixed-length messages. Using it directly on variable lengths allows forgery by extension. Standardized variants, such as CMAC, fix this problem.

3.29 Digital Signatures

A signature scheme is:

$$(\text{Gen}, \text{Sign}, \text{Vrfy}).$$

$$(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda),$$

$$\sigma \leftarrow \text{Sign}_{\text{sk}}(m), \quad \text{Vrfy}_{\text{pk}}(m, \sigma) \in \{0, 1\}.$$

Correctness:

$$\Pr[\text{Vrfy}_{\text{pk}}(m, \text{Sign}_{\text{sk}}(m)) = 1] = 1.$$

Typical security, EUF-CMA: after obtaining signatures of chosen messages, the adversary cannot produce a valid signature for a new message.

3.29.1 MAC versus Signature

MAC	Signature
shared key	public/private pair
any verifier can also forge	a public verifier must not be able to sign
does not prove authorship to third parties	allows public verifiability
usually faster	usually more expensive

“Non-repudiation” also depends on key protection, identity/certificate, policy and legal context; it is not produced by the cryptographic equation alone.

3.30 Types of Attack against Signatures

Attacks on signature schemes are classified along two axes: what the attacker can access, and what it manages to produce.

3.30.1 By the Attacker's Access

1. **key-only:** knows only the public key;
2. **known-message:** observes messages and signatures;
3. **chosen-message:** chooses messages to be signed;
4. **adaptive chosen-message:** chooses each query based on the previous answers.

3.30.2 By the Goal Achieved

1. **existential forgery:** produces a valid signature for some new message;
2. **selective forgery:** forges a chosen target message;
3. **universal forgery:** can sign any message;
4. **total break:** recovers the secret key.

EU-F-CMA already considers a single existential forgery after adaptive queries to be a win.

3.31 Textbook RSA Signature

The textbook RSA signature uses the same trapdoor as RSA encryption, with the roles of e and d swapped: only the key holder can produce a signature, and anyone can verify it.

Signature:

$$\sigma = m^d \bmod n.$$

Verification:

$$\sigma^e \stackrel{?}{\equiv} m \pmod{n}.$$

3.31.1 Example with small numbers

Reuse the key from the RSA encryption example, $n = 3233$, $e = 17$, $d = 2753$. To sign $m = 65$,

$$\sigma = m^d \bmod n = 65^{2753} \bmod 3233 = 588.$$

The verification returns

$$\sigma^e \bmod n = 588^{17} \bmod 3233 = 65 = m.$$

Compare with OAEP encryption: the same relation c^d and m^e underpins both uses, but with roles reversed. As a signature, randomness is not needed for whoever verifies; secure encoding is needed for whoever signs, otherwise the forgeries below apply.

3.31.2 Weaknesses

Existential forgery: choose an arbitrary σ and define

$$m = \sigma^e \bmod n.$$

Then (m, σ) verifies.

Multiplicative malleability: from signatures

$$\sigma_1 = m_1^d, \quad \sigma_2 = m_2^d,$$

it follows that:

$$\sigma_1 \sigma_2 \equiv (m_1 m_2)^d \pmod{n}.$$

In practice one uses hash-and-sign with secure encoding and domain separation, for example RSA-PSS. Merely replacing m by $H(m)$ without analyzing the encoding and the model does not automatically justify security.

3.32 ElGamal Signature

Parameters:

$$p \text{ prime,} \quad g \text{ generator of } \mathbb{Z}_p^*.$$

Secret and public key:

$$x \xleftarrow{\$} \{1, \dots, p-2\}, \quad y = g^x \bmod p.$$

To sign the digest $h = H(m)$:

1. choose a nonce

$$k \in \mathbb{Z}_{p-1}^*$$

such that $\gcd(k, p-1) = 1$;

2. compute

$$r = g^k \bmod p;$$

3. compute

$$s \equiv k^{-1}(h - xr) \pmod{p-1}.$$

The signature is (r, s) . Verification:

$$g^h \stackrel{?}{\equiv} y^r r^s \pmod{p}.$$

Correctness:

$$y^r r^s \equiv g^{xr} g^{ks} = g^{xr+ks} \equiv g^h \pmod{p},$$

since

$$ks \equiv h - xr \pmod{p-1}.$$

The nonce k must be secret, unpredictable and never reused.

3.32.1 Example with small numbers

Use $p = 23$, $g = 2$ and the secret key $x = 6$, hence

$$y = g^x = 2^6 \bmod 23 = 18.$$

For the digest $H(m) = 7$, choose the nonce $k = 5$; since $\gcd(5, 22) = 1$, it is valid. Then

$$r = g^k \bmod p = 2^5 \bmod 23 = 9.$$

Since $k^{-1} \equiv 9 \pmod{22}$,

$$s \equiv k^{-1}(H(m) - xr) \equiv 9(7 - 6 \cdot 9) \equiv 9 \cdot 19 \equiv 17 \pmod{22}.$$

The signature is $(r, s) = (9, 17)$. The verification computes

$$y^r r^s \bmod p = 18^9 \cdot 9^{17} \bmod 23 = 13,$$

$$g^{H(m)} \bmod p = 2^7 \bmod 23 = 13,$$

and accepts, since the two sides coincide. Note that here s was computed modulo $p-1 = 22$, and not modulo p ; confusing the two moduli is the most common mistake when reproducing the scheme.

3.33 DSA

DSA is a signature scheme over $\mathbb{Z}_p^*$ that works in a prime-order subgroup of order q . Working in the subgroup keeps the signature short while preserving security against generic discrete-log attacks.

Parameters:

- p a large prime;
- q a prime with $q \mid (p-1)$;
- g of order q in $\mathbb{Z}_p^*$.

Keys:

$$x \xleftarrow{\$} \mathbb{Z}_q^*, \quad y = g^x \bmod p.$$

3.33.1 Signature

Choose a nonce

$$k \xleftarrow{\$} \mathbb{Z}_q^*.$$

Compute:

$$r = (g^k \bmod p) \bmod q,$$

$$s = k^{-1}(H(m) + xr) \bmod q.$$

If $r = 0$ or $s = 0$, choose another k .

3.33.2 Verification

$$w = s^{-1} \bmod q,$$

$$u_1 = H(m)w \bmod q, \quad u_2 = rw \bmod q,$$

$$v = (g^{u_1}y^{u_2} \bmod p) \bmod q.$$

Accept if

$$v = r.$$

3.33.3 Example with small numbers

Choose $q = 11$ and $p = 23$, since $11 \mid 22$. Take $g = 2$, of order 11 in $\mathbb{Z}_{23}^*$, and $x = 3$, hence

$$y = g^x = 2^3 \bmod 23 = 8.$$

For $H(m) = 7$, choose $k = 5$. Then

$$r = (g^k \bmod p) \bmod q = (2^5 \bmod 23) \bmod 11 = 9 \bmod 11 = 9,$$

$$s = k^{-1}(H(m) + xr) \bmod q = 5^{-1}(7 + 3 \cdot 9) \bmod 11 = 9 \cdot 34 \bmod 11 = 9.$$

The verification computes

$$w = s^{-1} \bmod q = 9^{-1} \bmod 11 = 5,$$

$$u_1 = H(m)w \bmod q = 7 \cdot 5 \bmod 11 = 2, \quad u_2 = rw \bmod q = 9 \cdot 5 \bmod 11 = 1,$$

$$v = (g^{u_1}y^{u_2} \bmod p) \bmod q = (2^2 \cdot 8 \bmod 23) \bmod 11 = 9 \bmod 11 = 9.$$

Since $v = r = 9$, the signature is accepted. All the computation of s, u_1, u_2, v takes place modulo $q = 11$, and only the exponentiation $g^{u_1}y^{u_2}$ takes place modulo p ; it is exactly this separation that the optimization exploits.

3.33.4 Why is the optimization secure?

DSA works in the subgroup of prime order q , although the elements are represented modulo p . Exponents and signature components have a size related to q , reducing cost and size. The security of the generic attack on the DLP is governed by the order q :

$$O(\sqrt{q}).$$

Choosing q with an adequate size keeps the security even when $q \ll p$.

3.34 Nonce Reuse in DSA

If the same k signs m_1, m_2 , then r repeats:

$$s_1 = k^{-1}(H(m_1) + xr) \pmod{q},$$

$$s_2 = k^{-1}(H(m_2) + xr) \pmod{q}.$$

Subtracting:

$$s_1 - s_2 \equiv k^{-1}(H(m_1) - H(m_2)) \pmod{q}.$$

Therefore:

$$k \equiv (H(m_1) - H(m_2))(s_1 - s_2)^{-1} \pmod{q},$$

and then:

$$x \equiv (s_1 k - H(m_1))r^{-1} \pmod{q}.$$

A single repeated nonce compromises the entire private key.

3.35 ECDSA and Curve Standards

ECDSA carries the structure of DSA to the group of points of an elliptic curve. The multiplication

$$g^k$$

is replaced by the scalar multiplication

$$kG.$$

Known families include the NIST P curves, secp256k1, Curve25519 and Ed25519. There was historical concern about untransparent constants in some standardization processes. This does not amount to a public break of the NIST P curves. The best-known case of suspected backdoor was Dual_EC_DRBG, a curve-based pseudorandom generator later withdrawn; it is not the same as ECDSA.

Curve25519/Ed25519 are popular alternatives with publicly explained design and parameters, as well as formulas that facilitate implementations resistant to several classes of error.

4 Part VI — Exam-Oriented Review

4.1 What needs a definition, mechanism, and attack

Topic	Mandatory definition	Mechanism/calculation	Attack or limitation
PRG	indistinguishability	pseudo-OTP	enumeration is not PPT
CPA	game and advantage	encryption oracle	deterministic cipher
Feistel	round	forward and inversion	few rounds
AES	block cipher/SPN	four transformations	do not confuse block and key
CBC	chaining	Enc/Dec formulas	predictable IV/malleability
CTR	keystream by counter	XOR	repeated nonce
GCM	AEAD	CTR + GHASH	repeated nonce
Miller–Rabin	strong witness	sequence of squarings	false “probably prime”
RSA	textbook trapdoor permutation	Gen/Enc/Dec	determinism and malleability
OAEP	probabilistic encoding	two masks	incorrect implementation
DH	establishment	g^a, g^b, g^{ab}	MITM
ElGamal	PKE in a group	(g^y, mh^y)	repeated nonce/malleability
Pollard–rho	generic attack	collision	$O(\sqrt{q})$
Index Calculus	factor base	<i>smooth</i> relations	does not carry over to ECC
hash	three resistances	birthday	collision in $2^{n/2}$
CRC	generator polynomial	division in $GF(2)$	linear, not cryptographic
MAC	symmetric authentication	HMAC/CBC-MAC	naive construction
signature	EUF-CMA	Sign/Vrfy	nonce/textbook

Topic	Mandatory definition	Mechanism/calculation	Attack or limitation
DSA	signature in a prime-order subgroup	r, s reduced modulo q	nonce reuse

4.2 Confusions that need to be resolved

1. **Perfect versus computational secrecy:** exact equality for an unbounded adversary versus negligible advantage for PPT.
2. **PRG versus true randomness:** the supports are very different; only the efficient test is prevented.
3. **PRF versus PRP:** a random function can collide; a permutation cannot.
4. **Stream cipher versus CTR:** CTR turns a block cipher into a keystream.
5. **IV versus nonce:** the requirements depend on the mode; classic CBC asks for an unpredictable IV, while CTR/GCM demand uniqueness.
6. **Hash versus MAC:** a hash has no secret and does not authenticate origin.
7. **Collision versus second preimage:** in a collision the adversary chooses both inputs.
8. **Factoring versus RSA:** factoring breaks RSA; general equivalence is not proven.
9. **DLP versus DDH:** deciding whether a tuple is Diffie–Hellman can be easier than recovering the exponent in some groups.
10. **RSA correctness and $\gcd(m, n)$:** RSA remains correct; a message not coprime reveals a factor through the gcd.
11. **RSA encryption versus signature:** the exponents appear in the opposite order, but real security requires different encodings.
12. **Authenticity versus non-repudiation:** non-repudiation involves context beyond the algorithm.
13. **CRC versus hash:** CRC detects accidental error and is linear; a cryptographic hash is designed against an adversary.
14. **Modulus p versus $p - 1$ in the ElGamal signature:** r is computed modulo p , but s is computed modulo $p - 1$; in DSA, it is r and s that are reduced modulo q .

4.3 Formulas that must come without consulting

4.3.1 CBC

$$c_0 = IV, \quad c_i = E_k(m_i \oplus c_{i-1}), \quad m_i = D_k(c_i) \oplus c_{i-1}.$$

4.3.2 CTR

$$z_i = E_k(N \parallel \langle i \rangle), \quad c_i = m_i \oplus z_i.$$

4.3.3 Euler

$$\gcd(a, n) = 1 \implies a^{\varphi(n)} \equiv 1 \pmod{n}.$$

4.3.4 RSA

$$n = pq, \quad d = e^{-1} \bmod \varphi(n), \quad c = m^e \bmod n, \quad m = c^d \bmod n.$$

4.3.5 Diffie–Hellman

$$A = g^a, \quad B = g^b, \quad K = B^a = A^b = g^{ab}.$$

4.3.6 ElGamal

$$h = g^x, \quad (c_1, c_2) = (g^y, mh^y), \quad m = c_2/c_1^x.$$

4.3.7 Birthday

$$\Pr[\text{collision}] = 1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right) \approx 1 - e^{-q(q-1)/(2N)}.$$

4.3.8 ElGamal signature

$$r = g^k \bmod p, \quad s = k^{-1}(H(m) - xr) \bmod (p-1),$$

$$g^{H(m)} \stackrel{?}{=} y^r r^s \pmod{p}.$$

4.3.9 DSA

$$r = (g^k \bmod p) \bmod q, \quad s = k^{-1}(H(m) + xr) \bmod q.$$

4.4 Self-assessment questions

4.4.1 Foundations

1. Why do Lemma 2.5 and Definition 2.6 express the same goal?
2. Why is $1/\lambda^{100}$ not negligible?
3. How do you prove that the sum of two negligible functions is negligible?
4. How can two distributions be statistically far apart and computationally indistinguishable?
5. What are the five elements of a proof by reduction?
6. Why can deterministic encryption not be CPA-secure?

4.4.2 Symmetric

7. Why does the Feistel round function not need to be invertible?
8. Which components of AES guarantee inversion?
9. Why is DES obsolete even without a practical structural attack better than brute force?
10. Which requirement changes from CBC to CTR with respect to the IV/nonce?
11. Why can Chained CBC fail even though it looks like a single CBC message?
12. What does GCM authenticate besides the ciphertext?

4.4.3 Mathematics

13. How does extended Euclid produce a modular inverse?
14. When is “dividing” a congruence allowed?
15. What is the difference between the order of the group and the order of an element?
16. How do you verify that an element is a generator when the factorization of $|G|$ is known?
17. Why is Fermat not enough as a primality test?
18. What is the maximum error after t independent Miller–Rabin rounds?

4.4.4 Public key

19. Why does knowing $\varphi(n)$ allow factoring an RSA modulus $n = pq$?
20. Why is $e = 3$ dangerous in textbook RSA?
21. What is known and what is not known about factoring versus RSA inversion?
22. How does OAEP introduce randomization and all-or-nothing?
23. Why does plain DH not authenticate the parties?
24. How does ElGamal derive directly from DH?
25. Compare Pollard–rho and Index Calculus in time, memory, and applicable groups.
26. Why does ECC use smaller keys for the same classical strength?

4.4.5 Hashes and authentication

27. Differentiate preimage, second-preimage, and collision resistance.
28. Derive the birthday paradox approximation.
29. Why does a 256-bit output offer about 128 bits against collision?
30. Why should $H(k||m)$ not be adopted as a MAC without analysis?
31. Differentiate MAC and signature as to the people capable of verifying and forging.
32. Show two possible forgeries in textbook RSA signature.
33. Demonstrate the verification equation of the ElGamal signature.
34. Derive the recovery of k and x when DSA reuses a nonce.

4.5 Answers to Self-Assessment Questions

4.5.1 Foundations

4.6 Q1. Why do Lemma 2.5 and Definition 2.6 express the same goal?

Both statements say that the ciphertext reveals nothing about the plaintext; they differ only in the packaging of the statement. **Lemma 2.5** (the distributional characterization) says that for all $m, m' \in \mathcal{M}$ and all c ,

$$\Pr[\text{Enc}_K(m) = c] = \Pr[\text{Enc}_K(m') = c],$$

i.e. the ciphertext distribution does not depend on which message was encrypted. **Definition 2.6** (the game-based characterization) says that an adversary who chooses m_0, m_1 , receives $c \leftarrow \text{Enc}_K(m_b)$ for a uniform bit b , and outputs b' satisfies $\Pr[b' = b] = 1/2$ even with unlimited computation.

The two are equivalent because the game is just a particular way to *test* the distributional equality. If the distributions coincide, the pair (b, c) is independent: for any c the posterior $\Pr[b = 0 \mid c]$ equals the prior $1/2$, so no (even unbounded) adversary can beat $1/2$. Conversely, if some pair m_0, m_1 had $\Pr[\text{Enc}_K(m_0) = c] \neq \Pr[\text{Enc}_K(m_1) = c]$ for some c , that single c would distinguish them, and the adversary outputting 0 on c and 1 otherwise would attain $\Pr[b' = b] = 1/2 + \frac{1}{2} |\Pr[\text{Enc}_K(m_0) = c] - \Pr[\text{Enc}_K(m_1) = c]| > 1/2$. Hence the distributional statement and the game-based statement are two faces of the same condition — statistical independence of M and C — one stated with probabilities over ciphertexts and the other with an explicit winning probability.

Why the game form matters. Although equivalent for perfect secrecy, the game formulation is the one that *generalizes*: by changing the adversary’s power (unlimited $\rightarrow$ PPT), the available oracles (none $\rightarrow$ encryption/decryption), and the winning threshold ($= 1/2 \rightarrow \leq 1/2 + \text{negl}(\lambda)$), one obtains EAV (Def. 3.8), CPA, CCA, MAC and signature security as the *same* template. That is why the guide presents Lemma 2.5 as the mathematical content and Definition 2.6 as the reusable interface.

4.7 Q2. Why is $1/\lambda^{100}$ not negligible?

A function $\mu : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is negligible iff for **every** positive polynomial p there exists N with

$$\lambda \geq N \implies \mu(\lambda) < \frac{1}{p(\lambda)}.$$

The quantifier over polynomials is the crux: the function must beat the inverse of *every* polynomial, not merely

some polynomial. The polynomial $p(\lambda) = \lambda^{101}$ is positive and legitimate, and for every $\lambda \geq 2$,

$$\frac{1}{\lambda^{100}} > \frac{1}{\lambda^{101}},$$

so the required inequality $\mu(\lambda) < 1/p(\lambda)$ **fails** for all sufficiently large λ . (Verified: $\frac{1}{\lambda^{100}} < \frac{1}{\lambda^{101}}$ is **False** for $\lambda \in \{2, 3, 10, 1000\}$.) A fit bound $\mu(\lambda) < 1/\lambda^{100}$ only rules out polynomials of degree ≤ 100 ; a larger polynomial defeats it. This is exactly the distinction between “smaller than some polynomial inverse” (which $1/\lambda^{100}$ satisfies) and “smaller than every polynomial inverse” (which it does not). By contrast $2^{-\lambda}$ beats every polynomial, since $2^{-\lambda}\lambda^c \rightarrow 0$ for each fixed c , so it is negligible.

4.8 Q3. How do you prove that the sum of two negligible functions is negligible?

Let μ_1, μ_2 be negligible and set $\mu = \mu_1 + \mu_2$. We must show that for every fixed $c > 0$ there is N with $\mu(\lambda) < \lambda^{-c}$ for all $\lambda \geq N$.

Fix $c > 0$. By negligibility of each function, there exist N_1, N_2 such that

$$\lambda \geq N_1 \implies \mu_1(\lambda) < \frac{1}{2\lambda^c}, \quad \lambda \geq N_2 \implies \mu_2(\lambda) < \frac{1}{2\lambda^c}.$$

Take $N = \max(N_1, N_2)$. For every $\lambda \geq N$ both bounds hold simultaneously, so

$$\mu(\lambda) = \mu_1(\lambda) + \mu_2(\lambda) < \frac{1}{2\lambda^c} + \frac{1}{2\lambda^c} = \frac{1}{\lambda^c}.$$

Since $c > 0$ was arbitrary, μ beats $1/p$ for every polynomial p , i.e. μ is negligible. (Verified with $c = 3$: the two halves sum to exactly $1/\lambda^3$.)

The halving trick is the whole idea: split the target budget evenly so each function is allowed a fixed fraction of it. The same argument extends by induction to any *constant* number of negligible terms. It also shows that a polynomial $p(\lambda)$ times a negligible function stays negligible: p only consumes a fixed power λ^k , so absorb λ^k into the constant c in the definition.

4.9 Q4. How can two distributions be statistically far apart and computationally indistinguishable?

Statistical distance and computational indistinguishability measure different adversaries. Take the PRG example with stretch $\ell(\lambda) = 2\lambda$. If $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$, then $|\text{Im}(G)| \leq 2^\lambda$, while $|\{0, 1\}^{2\lambda}| = 2^{2\lambda}$. Hence a uniform string falls in $\text{Im}(G)$ with probability at most

$$\frac{2^\lambda}{2^{2\lambda}} = 2^{-\lambda},$$

whereas an output of G lies in $\text{Im}(G)$ with probability 1. Therefore the **statistical distance** between the two distributions is at least $1 - 2^{-\lambda}$, essentially maximal (verified numerically: for $\lambda = 10$ this equals $1 - 2^{-10} \approx 0.99902$).

The two ensembles are nonetheless **computationally indistinguishable** because testing membership in $\text{Im}(G)$ requires enumerating all 2^λ seeds and evaluating G on each — a step exponential in λ . A computationally unbounded distinguisher can do it and wins almost always, but no PPT distinguisher can: for every PPT D ,

$$|\Pr[D(G(s)) = 1] - \Pr[D(r) = 1]| \leq \text{negl}(\lambda).$$

So “statistically far” is a statement about the two distributions as static objects, while “computationally indistinguishable” is a statement about the power of efficient algorithms to tell them apart. The pseudo-OTP inherits exactly this gap: $m \oplus G(k)$ is statistically close to no shift if the pad were random, but the image-restricted pad is computationally hidden, giving EAV security despite a statistically detectable artifact. This is the entire purpose of the asymptotic definition: it replaces an impossible perfect-secrecy requirement with a feasible computational one.

4.10 Q5. What are the five elements of a proof by reduction?

A reduction proves “ $\mathcal{A}$ breaks $\Pi \Rightarrow \mathcal{B}^{\mathcal{A}}$ breaks assumption H ” through five mandatory elements:

1. **Assumption (the hypothesis to violate).** State that there exists an adversary $\mathcal{A}$ against Π with non-negligible advantage; this is the starting point to be contradicted.
2. **Construction of the reducer $\mathcal{B}$.** Give an explicit algorithm: $\mathcal{B}$ receives an instance of problem H (e.g. a string w that is either $G(k)$ or uniform) and uses $\mathcal{A}$ as a subroutine.
3. **Simulation.** Show that the view $\mathcal{B}$ presents to $\mathcal{A}$ has exactly the distribution $\mathcal{A}$ expects in the real security game, so $\mathcal{A}$ ’s behavior is unchanged.
4. **Case analysis and computation of the advantage.** Analyze the two worlds (real vs. random/fake) and derive a relation such as

$$\text{Adv}_{\mathcal{B}}^H = |\Pr[b' = b \mid \Pi] - \frac{1}{2}| = \text{Adv}_{\mathcal{A}}^\Pi,$$

using that in the fake world the simulation is a perfect OTP and the probability is exactly $1/2$.

5. **Contradiction.** Since $\mathcal{B}$ is efficient whenever $\mathcal{A}$ is, a non-negligible $\text{Adv}_{\mathcal{A}}^\Pi$ would give $\mathcal{B}$ a non-negligible advantage against H , contradicting the assumed security of H .

In the guide’s six-step listing, step 4 is split into “case analysis” and “advantage relation”; grouped as above they are the five conceptual elements. The three details that must always be checked are that $\mathcal{B}$ is efficient, that the real-world simulation is perfect, and that the ideal-world probability is exactly $1/2$ (rather than merely “hard to distinguish”).

4.11 Q6. Why can deterministic encryption not be CPA-secure?

In the CPA game the adversary has an encryption oracle $m \mapsto \text{Enc}_k(m)$ before and after the challenge. If Enc is deterministic, the oracle is a deterministic function of the plaintext, and the adversary wins outright:

1. choose two distinct messages $m_0 \neq m_1$ of equal length;
2. query $c_0 \leftarrow \text{Enc}_k(m_0)$;
3. receive the challenge $c^* = \text{Enc}_k(m_b)$;
4. output $b' = 0$ if $c^* = c_0$ and $b' = 1$ otherwise.

This always succeeds. Determinism plus **correctness** makes encryption injective on valid messages: if two distinct valid messages $m_0 \neq m_1$ encrypted to the same ciphertext, distinctness would force $\text{Dec}_k(c) = m_0$ and $\text{Dec}_k(c) = m_1$ simultaneously, contradicting correctness. Hence $c^* = c_0$ holds **iff** $m_b = m_0$, i.e. iff $b = 0$. Therefore

$$\Pr[b' = b] = 1, \quad \text{Adv}_{\Pi, \mathcal{A}}^{\text{cpa}} = |1 - \tfrac{1}{2}| = \tfrac{1}{2},$$

which is non-negligible (a constant), so the scheme fails CPA security.

The fix is to make each encryption depend on fresh randomness, a unique nonce, or evolving state, so that encrypting the same plaintext twice yields different ciphertexts. This is why probabilistic encryption (e.g. random-pad pseudo-OTP) and modes with random IVs/nonces are required: without them, the equality test above is a certificate of insecurity. Note that a deterministic cipher *can* still be EAV-secure — the failure is specifically caused by the oracle access granted in CPA.

4.11.1 Symmetric

4.12 Q7. Why does the Feistel round function not need to be invertible?

In a Feistel network the block is split into halves (L_0, R_0) and round i computes

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F_i(R_{i-1}).$$

Inversion does not require knowing F_i^{-1} : because the new left half is literally the old right half, we recover $R_{i-1} = L_i$ by inspection, and then

$$L_{i-1} = R_i \oplus F_i(R_{i-1}) = R_i \oplus F_i(L_i).$$

The correction works purely by the XOR cancellation $x \oplus y \oplus y = x$ (characteristic 2), so F_i is evaluated *forward* in both directions and may be any function whatsoever — non-injective, non-surjective, arbitrary. This is why Feistel can use a one-way, keyed round function (no need for a public closed-form inverse), and it is the reason DES could be built from S-boxes without ever specifying an inverse S-box. To undo the whole network one reuses the same round machinery with the subkeys in reverse order k_4, k_3, k_2, k_1 , since the structure is symmetric. The price is that the round function is only half-block mixing per round, so more rounds are needed to reach the same diffusion as a substitution-permutation network.

4.13 Q8. Which components of AES guarantee inversion?

Decryption in AES is possible because every individual transformation of the state is a bijection, unlike a Feistel cipher that deliberately avoids that requirement:

- **SubBytes** is a bitwise permutation: the S-box is built from the multiplicative inverse in $GF(2^8)$ (modulus $x^8 + x^4 + x^3 + x + 1$) followed by an affine bit transformation, and both maps are bijections (zero is mapped to zero before the affine map), so an inverse S-box exists.
- **ShiftRows** cyclically shifts rows by 0, 1, 2, 3; it is inverted by shifting the opposite way (or by 0, 3, 2, 1).
- **MixColumns** multiplies each column by a fixed matrix over $GF(2^8)$ whose determinant is nonzero, hence an invertible linear map; it is inverted by the inverse matrix.
- **AddRoundKey** is its own inverse, because $(x \oplus k) \oplus k = x$.

Because all four operations are invertible, decryption applies their inverses in reverse order — conceptually InvShiftRows , InvSubBytes , AddRoundKey , InvMixColumns — and AES never needs the Feistel “invertibility from non-invertible parts” trick. The only remaining ingredient is that the subkeys are not secret guesses but are regenerated deterministically by the key schedule and used from the last one back to the first. (The last encryption round omits MixColumns , and correspondingly the first decryption round omits InvMixColumns .)

4.14 Q9. Why is DES obsolete even without a practical structural attack better than brute force?

DES remains a cipher with a remarkably robust design: sixteen Feistel rounds, and decades of structural cryptanalysis (differential, linear) never produced a break faster than exhaustive search in practice. Its obsolescence is a *sizing* failure, not a structure break. The nominal key is 64 bits but 8 are parity bits, leaving only 56 effective bits, so exhaustive search costs about 2^{55} trials on average — trivially within reach of specialized hardware (and was already broken by EFF’s Deep Crack and distributed efforts). Second, the block is only 64 bits: by the birthday bound, collisions among blocks appear after roughly 2^{32} blocks, so a single key cannot

safely encrypt more than a few tens of gigabytes, which is far too small for modern traffic volumes. The lesson the guide draws is that a cipher can be well studied and structurally sound and still fail because the key and block became small. Three-key TripleDES stretches the key to a nominal 168 bits but only buys about 112 bits of classical security (meet-in-the-middle reductions) and keeps the 64-bit block, so it is a stopgap rather than a fix; AES, with a 128-bit block and 128/192/256-bit keys, is the actual remedy.

4.15 Q10. Which requirement changes from CBC to CTR with respect to the IV/nonce?

Both modes need a value that is used once per encryption, but the security requirement is different in kind. CBC needs an IV that is **unpredictable** (uniform and secret-independent, but not repeatable): the standard definition is “uniform and unpredictable.” Predictability is fatal because CBC yields $c_1 = E_k(m_1 \oplus IV)$, so an adversary who can predict or control the IV can mount the classical chosen-plaintext distinguishing attack; unpredictability is precisely what defeats it, while the IV may still be public *after* it is chosen. CTR only needs a value that is **unique** — a nonce that never repeats under the same key: the keystream is $z_i = E_k(N \parallel \langle i \rangle)$, so what must never happen is that the pair (k, N) recurs, otherwise two messages share the keystream and $c \oplus c' = m \oplus m'$. CTR’s nonce need not be secret and, crucially, need not be unpredictable: a simple counter that increments monotonically is perfectly secure, since uniqueness alone suffices. In short, CBC requires unpredictability (fresh randomness is the easy way to obtain it), whereas CTR requires non-repetition (a deterministic counter is the canonical way). Neither mode, however, provides integrity by itself.

4.16 Q11. Why can Chained CBC fail even though it looks like a single CBC message?

Chained CBC uses the last ciphertext block of the previous encrypted message as the IV of the next message, so the sequence superficially resembles one long CBC encryption. The flaw is that this “IV” is a value the adversary already knows *before* choosing the next message, which is exactly what the CPA security definition forbids (the IV must be chosen independently of the message and unknown in advance). Concretely, suppose a query encrypts block X with IV V , giving $C = E_k(X \oplus V)$; in Chained CBC, C becomes the public IV of the next message. In the challenge the adversary submits

$$M_0 = X \oplus V \oplus C, \quad M_1 \neq M_0.$$

If the challenge bit is 0, the first ciphertext block is

$$C^* = E_k(M_0 \oplus C) = E_k(X \oplus V) = C,$$

so the adversary outputs $b' = 0$ when $C^* = C$; if $b' = 1$ otherwise, and for $M_1 \neq M_0$ the bijectivity of E_k guarantees $C^* \neq C$. The adversary is therefore always correct, so Chained CBC cannot be CPA-secure. The essential point is that a public IV is fine once it has been chosen, but a *predictable* IV that is known before the message is chosen breaks the security definition — “looking like a concatenation of CBC messages” is not a substitute for the adaptive-model requirement.

4.17 Q12. What does GCM authenticate besides the ciphertext?

GCM is an AEAD mode that authenticates the ciphertext **and the associated data (AAD)** — plus, implicitly, the lengths and the nonce-derived framing. It combines CTR-mode encryption for confidentiality with the GHASH universal hash for integrity: with $H = E_k(0^{128})$, GHASH accumulates the AAD blocks $A_1, \dots, A_u$, the ciphertext blocks $C_1, \dots, C_v$, and a final block $\text{len}(A) \parallel \text{len}(C)$, and the tag is

$$T = \text{MSB}_t(E_k(J_0) \oplus \text{GHASH}_H(A, C)).$$

The final length block binds the lengths so that concatenation ambiguities cannot be exploited, and J_0 is derived from the nonce, so the tag also implicitly binds the nonce. The receiver recomputes T and accepts only if the comparison matches (done without timing leakage). The AAD is data that must remain visible and unmodified but need not be secret — typical examples are packet headers, routing information, or protocol metadata. Thus any change to the nonce, AAD, ciphertext, tag, or lengths causes rejection except with the small forgery probability allowed by the tag length; and reusing a nonce in GCM is catastrophic because it both repeats the CTR keystream and creates algebraic relations in the GHASH polynomial that destroy authentication.

4.17.1 Mathematics

4.18 Q13. How does extended Euclid produce a modular inverse?

The **extended Euclidean algorithm** runs the ordinary Euclidean algorithm while additionally tracking, at every step, the representation of each remainder as a linear combination of the original inputs. Given a, n with $\gcd(a, n) = 1$, it returns integers x, y satisfying Bézout’s identity

$$ax + ny = \gcd(a, n) = 1.$$

The mechanism is that the elementary division step $r_{i+1} = r_{i-1} - q_i r_i$ preserves linear combinations: if $r_{i-1} = s_{i-1}a + t_{i-1}n$ and $r_i = s_i a + t_i n$, then

$$r_{i+1} = r_{i-1} - q_i r_i = (s_{i-1} - q_i s_i) a + (t_{i-1} - q_i t_i) n.$$

So the algorithm carries the triples (r_i, s_i, t_i) initialized as $(a, 1, 0)$ and $(n, 0, 1)$ and updates them in lockstep; the last nonzero remainder is $\gcd(a, n) = 1$, and the coefficients on that row are (x, y) . Reducing $ax + ny = 1$ modulo n kills the ny term, giving

$$ax \equiv 1 \pmod{n}, \quad \text{hence} \quad a^{-1} \equiv x \pmod{n}.$$

Worked case: inverse of 17 modulo 101. The divisions are

$$101 = 5 \cdot 17 + 16, \quad 17 = 1 \cdot 16 + 1.$$

Substituting backwards,

$$1 = 17 - 16 = 17 - (101 - 5 \cdot 17) = 6 \cdot 17 - 101.$$

Here $x = 6$, $y = -1$, and indeed $17 \cdot 6 + 101 \cdot (-1) = 102 - 101 = 1$. Therefore

$$17^{-1} \equiv 6 \pmod{101} \quad (\text{check: } 17 \cdot 6 = 102 \equiv 1 \pmod{101}).$$

Second case: inverse of 3 modulo 11. $11 = 3 \cdot 3 + 2$ and $3 = 1 \cdot 2 + 1$, so $1 = 3 - 2 = 3 - (11 - 3 \cdot 3) = 4 \cdot 3 - 11$, giving $3^{-1} \equiv 4 \pmod{11}$ (check: $3 \cdot 4 = 12 \equiv 1$).

4.19 Q14. When is ‘dividing’ a congruence allowed?

Given $a \equiv b \pmod{n}$ and $c \equiv d \pmod{n}$, addition and multiplication are always valid: $a + c \equiv b + d$ and $ac \equiv bd \pmod{n}$. **Division is the exception:** a factor a may be cancelled from both sides only when a is invertible modulo n , that is, when

$$\gcd(a, n) = 1.$$

In that case multiplying by $a^{-1} \pmod{n}$ is legitimate and $ax \equiv ay \pmod{n}$ yields $x \equiv y \pmod{n}$. When $\gcd(a, n) = g > 1$ the factor has no inverse and cancellation is **not** allowed as stated: the correct general rule is

$$ax \equiv ay \pmod{n} \iff x \equiv y \pmod{\frac{n}{g}}, \quad g = \gcd(a, n).$$

Intuitively, reducing the modulus by g is exactly what makes the reduced factor invertible modulo the reduced modulus. The clearest counterexample is

$$2x \equiv 2 \pmod{6}.$$

Here $\gcd(2, 6) = 2 \neq 1$. Both $x = 1$ and $x = 4$ satisfy the congruence ($2 \cdot 1 = 2$, $2 \cdot 4 = 8 \equiv 2 \pmod{6}$), so cancelling the 2 to conclude $x \equiv 1 \pmod{6}$ would be wrong and would *lose the solution* $x = 4$. Applying the rule with $g = 2$ gives the correct reduction modulus $6/2 = 3$, and indeed

$$2x \equiv 2 \pmod{6} \iff x \equiv 1 \pmod{3},$$

whose solutions modulo 6 are $\{1, 4\}$. By contrast, in $5x \equiv 5 \pmod{7}$ we have $\gcd(5, 7) = 1$, so cancellation is legal and $x \equiv 1 \pmod{7}$ is the unique solution.

4.20 Q15. What is the difference between the order of the group and the order of an element?

The **order of a group** $(G, \circ)$ is its cardinality $|G|$ — how many elements it contains. The **order of an element** $g \in G$ is the smallest positive integer r such that

$$g^r = e,$$

where e is the identity. They are different kinds of quantity: $|G|$ is a property of the whole set, whereas $\text{ord}(g)$ is a property of one element, and it need not divide, equal, or even relate obviously to another element’s order. They are linked by **Lagrange’s theorem**, which states

$$\text{ord}(g) \mid |G|;$$

this is why the order of any element must be a divisor of the group order, and why the group can only be cyclic (have an element of order $|G|$) when some divisor $|G|$ is actually attained.

Additive example $\mathbb{Z}_{36}^+$. The group order is $|G| = 36$. For an element a its order is $36/\gcd(a, 36)$: e.g. $\text{ord}(1) = 36/\gcd(1, 36) = 36$ (so 1 is a generator), $\text{ord}(9) = 36/\gcd(9, 36) = 36/9 = 4$, and $\text{ord}(18) = 36/\gcd(18, 36) = 36/18 = 2$. All three orders (36, 4, 2) divide $|G| = 36$, as Lagrange requires, yet they are distinct from one another and from 36.

Multiplicative example $\mathbb{Z}_{36}^*$. Here $|G| = \varphi(36) = 12$, but the element orders that occur are only 1, 2, 3, 6 — no element has order 12. Thus the group order is 12 while the largest element order is $6 < 12$, which is exactly why $\mathbb{Z}_{36}^*$ is **not cyclic**. In $\mathbb{Z}_{37}^*$ instead $|G| = 36$ and some elements (e.g. 2) do have order $36 = |G|$, so the group is cyclic.

4.21 Q16. How do you verify that an element is a generator when the factorization of $|G|$ is known?

Let G be a cyclic group of known order $N = |G|$, and let

$$N = \prod_{i=1}^t p_i^{e_i}$$

with $p_1, \dots, p_t$ the **distinct** prime factors of N . The generator test is

$$g \text{ generates } G \iff g^{N/p_i} \neq e \text{ for every } i = 1, \dots, t.$$

The reason is the following. Since $\text{ord}(g) \mid N$, the order is some divisor of N . The order fails to be N exactly when at least one prime factor can be dropped, i.e. when $\text{ord}(g) \mid N/p_i$ for some i ; but $\text{ord}(g) \mid N/p_i$ is equivalent to $g^{N/p_i} = e$. So if some test yields the identity, the order divides the strictly smaller number N/p_i and g is not a generator. If **none** of the tests yields the identity, then no prime factor p_i can be removed from N , so $\text{ord}(g) = N$ and g is a generator. Only the distinct primes are tested, not all divisors, which makes the test

efficient; it does, however, require the factorization of N to be known.

Worked case: $\mathbb{Z}_{37}^*$, $N = 36$. Factor

$$36 = 2^2 \cdot 3^2,$$

so the distinct primes are 2 and 3 and the two exponents to test are $N/2 = 18$ and $N/3 = 12$. - Candidate $g = 2$: $2^{18} \bmod 37 = 36 \neq 1$ and $2^{12} \bmod 37 = 26 \neq 1$. Both tests pass, so $\text{ord}(2) = 36$ and **2 is a generator** (the guide lists it among the generators of $\mathbb{Z}_{37}^*$). - Candidate $g = 3$: $3^{18} \bmod 37 = 1$. The test fails, so $\text{ord}(3) \mid 18$ — in fact $\text{ord}(3) = 18$ — and 3 is **not** a generator.

A prime-order group is the easy special case: if $|G| = q$ is prime then the only distinct factor is q , the single test is $g^{q/q} = g^1 \neq e$, and every non-identity element automatically passes.

4.22 Q17. Why is Fermat not enough as a primality test?

Fermat's theorem says that if p is prime and $a \not\equiv 0 \pmod{p}$, then $a^{p-1} \equiv 1 \pmod{p}$. Read as a test, this is only **one-sided**: if for some base a one finds

$$a^{n-1} \not\equiv 1 \pmod{n},$$

then n is certainly composite (that a is a *Fermat witness*). The converse fails, so passing the test does not certify primality. Composite numbers that pass for a given base are called **Fermat pseudoprimes**; for example

$$341 = 11 \cdot 31 \text{ is composite, yet } 2^{340} \equiv 1 \pmod{341},$$

so base 2 fails to expose it. Worse, there are **Carmichael numbers**: composites that satisfy $a^{n-1} \equiv 1 \pmod{n}$ for *every* base a with $\gcd(a, n) = 1$. The smallest is

$$561 = 3 \cdot 11 \cdot 17,$$

and one checks that all $\varphi(561) = (3-1)(11-1)(17-1) = 320$ coprime bases pass; there is no base with $\gcd(a, 561) = 1$ that detects its compositeness. So Fermat cannot bound how often it errs: the fraction of useless bases can be 1 for Carmichael numbers, meaning repeated rounds give no error decay at all. The remedy is a stronger test such as **Miller–Rabin**, which rules out the $n-1 = 2^s d$ squaring chain and therefore has a proven $1/4$ bound on the fraction of fooling bases. Indeed 341, which passes Fermat in base 2, is **rejected** by Miller–Rabin in base 2, illustrating that the extra structure strictly increases detection power.

4.23 Q18. What is the maximum error after t independent Miller–Rabin rounds?

In Miller–Rabin one writes $n-1 = 2^s d$ with d odd, picks a random base $a \in \{2, \dots, n-2\}$, and checks whether a^d or one of its $s-1$ successive squares reaches $\equiv -1 \pmod{n}$ along the chain; a base that makes a composite pass is called a **strong liar**. The key theorem is: *if n is an odd composite, then at most $1/4$ of the bases in $\{2, \dots, n-2\}$ are strong liars*. Therefore the probability that a **fixed** composite n survives one random round is at most $1/4$, and the probability that it survives all t **independent** rounds (fresh, independently random bases each time) is at most

$$\Pr[\text{composite accepted as prime}] \leq \left(\frac{1}{4}\right)^t = 4^{-t}.$$

Independence is what turns the per-round bound into the product 4^{-t} ; if the same base were reused it would give no improvement. The bound is a worst-case, universal guarantee — it holds for every odd composite n without knowing anything about the density of primes.

Numeric values.

$$4^{-t} = 2^{-2t} :$$

$t = 1$ gives $2^{-2} = 1/4 = 0.25$; $t = 5$ gives $2^{-10} \approx 9.77 \times 10^{-4}$; $t = 10$ gives $2^{-20} \approx 9.54 \times 10^{-7}$; $t = 20$ gives $2^{-40} \approx 9.09 \times 10^{-13}$; $t = 40$ gives $2^{-80} \approx 8.27 \times 10^{-25}$; $t = 64$ gives $2^{-128} \approx 2.94 \times 10^{-39}$.

Two caveats the guide stresses: this is the probability for a **previously fixed** composite, not the chance that a number returned by a prime generator is composite — the latter is a *conditional* probability that also depends on the prior density of primes among the candidates and can be much smaller (e.g. HAC Table 4.3 gives $\approx 2^{-19}$ for one round near 300 bits, $\approx 2^{-28}$ near 350 bits). And Miller–Rabin is probabilistic only in its acceptance direction: when it declares “composite”, the conclusion is certain. Note also that 4^{-t} is a proved ceiling; for many specific composites the true fraction of liars is strictly smaller.

4.23.1 Public key

4.24 Q19. Why does knowing $\varphi(n)$ allow factoring an RSA modulus $n = pq$?

For $n = pq$ with p, q prime, the totient is $\varphi(n) = (p-1)(q-1) = pq - p - q + 1 = n - (p+q) + 1$. Thus knowing n and $\varphi(n)$ yields the **sum** of the primes for free:

$$p + q = n - \varphi(n) + 1.$$

Since we also know their **product** $pq = n$, the pair (p, q) are exactly the two roots of the quadratic

$$x^2 - (p+q)x + pq = 0 \iff x^2 - (n - \varphi(n) + 1)x + n = 0.$$

The discriminant is $\Delta = (p+q)^2 - 4n = (p-q)^2$, a perfect square, so

$$p = \frac{(p+q) + \sqrt{\Delta}}{2}, \quad q = \frac{(p+q) - \sqrt{\Delta}}{2},$$

and both are integers. So $\varphi(n)$ is *equivalent* to the factorization: given either one, the other is recovered in polynomial time. (Conversely, $p, q \Rightarrow \varphi(n)$ is immediate.)

Worked case (from the guide): $n = 4294049777$, $\varphi(n) = 4293918720$. Then $p + q = n - \varphi(n) + 1 = 131058$ and $\Delta = 131058^2 - 4 \cdot 4294049777 = 256 = 16^2$, so $p = (131058 + 16)/2 = 65537$ and $q = (131058 - 16)/2 = 65521$. I verified $65537 \cdot 65521 = 4294049777$ and $(65536)(65520) = 4293918720$. A tiny analogue: $n = 3233$, $\varphi(n) = 3120$ gives $p + q = 114$, $\Delta = 114^2 - 4 \cdot 3233 = 64 = 8^2$, hence $p = 61$, $q = 53$.

Because $d \equiv e^{-1} \pmod{\varphi(n)}$, anyone who learns $\varphi(n)$ can compute the private exponent and break RSA — this is exactly why the factorization of n must stay secret.

4.25 Q20. Why is $e = 3$ dangerous in textbook RSA?

The map is $c = m^e \bmod n$ with no random padding, so three concrete weaknesses appear:

1. **Direct integer cube root.** If the message is small enough that $m^3 < n$, then the modular reduction never happens and $c = m^3$ as an ordinary integer. Taking the real cube root recovers m with no key at all. Example: $n = 3233$, $m = 12$ gives $c = 12^3 = 1728 < 3233$, and $\sqrt[3]{1728} = 12$ (verified).
2. **Broadcast (Håstad) attack.** If the same m is sent to three recipients with pairwise coprime moduli n_1, n_2, n_3 and $m^3 < n_1 n_2 n_3$, the CRT reconstructs the integer m^3 exactly, and a cube root yields m . Small case verified: $m = 7$, moduli 3233, 3239, 3251; the residues of $7^3 = 343$ combine to give 343 modulo $M = 34043454437$, and $\sqrt[3]{343} = 7$.
3. **Algebraic malleability.** Low exponents leave more exploitable structure (related-message, small-message and padding errors) because the multiplicative homomorphism $\text{Enc}(m_1)\text{Enc}(m_2) = \text{Enc}(m_1 m_2)$ is unchecked by randomness.

The fix is never “choose a bigger e blindly” but to use a proper randomized encoding: with OAEP (or a correct, analyzed padding), $e = 3$ is not automatically insecure. $e = 65537 = 2^{16} + 1$ is preferred because it is efficient to exponentiate, prime enough for $\gcd(e, \varphi(n)) = 1$, and far from the tiny-exponent pitfalls.

4.26 Q21. What is known and what is not known about factoring versus RSA inversion?

Known (one direction): factoring implies RSA inversion. Given p, q one computes $\varphi(n) = (p - 1)(q - 1)$, then $d \equiv e^{-1} \bmod \varphi(n)$, and decrypts any ciphertext. So

$$\text{factor } n \implies \text{invert RSA.}$$

Also known: giving away $\varphi(n)$ factors n (Q19), and if any message satisfies $\gcd(m, n) \neq 1$ then computing that gcd immediately reveals p or q .

Not known (the other direction): it is *not* proven that inverting RSA requires factoring. There is no reduction showing that an efficient RSA-inversion algorithm must yield a factorization of n ; even the weaker statement “RSA is as hard as factoring” is only a widely believed heuristic, not a theorem. It remains conceivable that the **RSA problem is strictly easier** than factoring (e.g. a hypothetical inversion oracle might not expose p, q).

Correct framing: security of RSA rests on the *assumption* that the RSA problem (and/or factoring) is hard for the chosen parameters. One should therefore write

$$\text{factoring } n \Rightarrow \text{invert RSA,}$$

and *never* assert the converse as an established equivalence. Known results do show that for special small exponents an inversion oracle can help factor, but no general equivalence is proved.

4.27 Q22. How does OAEP introduce randomization and all-or-nothing?

OAEP is a Feistel-like pre-processing applied *before* the RSA map. For an encoded message M and a fresh random seed r :

$$X = M \oplus G(r), \quad Y = r \oplus H(X),$$

and RSA is applied to the block $X\|Y$. Decoding reverses it using $r = Y \oplus H(X)$ and $M = X \oplus G(r)$, where G, H are hash functions.

Randomization: the seed $r \xleftarrow{\$} \{0, 1\}^k$ is chosen anew for every encryption, so even a fixed message M yields different $X\|Y$ and hence different ciphertexts. Encryption becomes probabilistic, defeating the determinism that lets a CPA adversary test a guessed message by re-encrypting it. Randomness is *recovered* during decryption rather than transmitted, so no separate seed field is needed.

All-or-nothing: because X depends on all of r through $G(r)$ and Y depends on all of X through $H(X)$, every output bit depends on every input bit. Corrupting or omitting *any* part of the block destroys the ability to recover *either* the seed or the message — there is no partial information leakage. This also frustrates the multiplicative malleability of textbook RSA, since valid encodings are not closed under multiplication. Real implementations add a label, a hash of the label, and precise encoding rules; a hand-invented “similar” padding is not a substitute.

4.28 Q23. Why does plain DH not authenticate the parties?

Plain Diffie–Hellman exchanges only unauthenticated group elements: Alice sends $A = g^a$, Bob sends $B = g^b$, and each computes $K = g^{ab}$. Nothing binds these values to an identity, so an active adversary Mallory can interpose a full man-in-the-middle:

1. Alice sends g^a ; Mallory intercepts it, keeps it, and forwards g^{m_1} to Bob.
2. Bob sends g^b ; Mallory keeps it and forwards g^{m_2} to Alice.
3. Alice computes $K_{AM} = g^{am_1}$; Bob computes $K_{MB} = g^{bm_2}$; Mallory computes both g^{am_1} and g^{bm_2} .

Mallory now shares one key with Alice and a *different* key with Bob, decrypting, reading (or modifying), and re-encrypting every message. Neither party can detect this, because the received values g^{m_1}, g^{m_2} are perfectly valid group elements. DH is therefore secure only against a **passive** eavesdropper (under CDH); it provides no authentication. To stop MITM the ephemeral values must be authenticated — by signatures, certificates, or a previously shared key (giving STS/authenticated DH).

4.29 Q24. How does ElGamal derive directly from DH?

ElGamal is DH with one party’s contribution made *static* (the public key) and the other *ephemeral* (the randomness of a single encryption). In $G = \langle g \rangle$ of order q , the receiver picks $x \xleftarrow{\$} \mathbb{Z}_q$ and publishes $h = g^x$; this fixed h is the receiver’s DH contribution, playing the role of g^a . To encrypt $m \in G$, the sender chooses a fresh $y \xleftarrow{\$} \mathbb{Z}_q$ and sends

$$c_1 = g^y, \quad c_2 = m \cdot h^y.$$

Here g^y is precisely the sender’s DH message g^b , and $h^y = (g^x)^y = g^{xy}$ is the **Diffie–Hellman shared secret**, used as a one-time pad on the message. Decryption recomputes the same secret from the other side:

$$m = \frac{c_2}{c_1^x} = \frac{m \cdot (g^x)^y}{(g^y)^x} = m.$$

So the receiver, holding x , can form g^{xy} from the sender’s public g^y , exactly as in DH — the roles of “ g^a fixed / g^b ephemeral” are simply labeled pk/ c_1 .

Worked case ($p = 23, g = 5$, matching the guide): key $x = 6$, so $h = 5^6 \bmod 23 = 8$. Encrypt $m = 7$ with $y = 3$: $c_1 = 5^3 \bmod 23 = 10$ and $c_2 = 7 \cdot 8^3 \bmod 23 = 7 \cdot 6 = 19$, so $c = (10, 19)$. Decrypt: $c_1^x = 10^6 \bmod 23 = 6$ (verified) and $6^{-1} \equiv 4 \pmod{23}$, so $m = 19 \cdot 4 \bmod 23 = 7$ (verified). A fresh y per message is essential: reusing it gives $c_2/c_2' = m/m'$, exposing the plaintext ratio. With fresh randomness and DDH, ElGamal is CPA-secure.

4.30 Q25. Compare Pollard–rho and Index Calculus in time, memory, and applicable groups.

Pollard–rho for the DLP. A *generic* collision-based method: iterates states $X_i = g^{a_i} h^{b_i}$; a collision $X_i = X_j$ yields $g^{a_i+b_i x} = g^{a_j+b_j x}$, hence the linear congruence

$$(b_i - b_j)x \equiv a_j - a_i \pmod{q},$$

solved as $x \equiv (a_j - a_i)(b_i - b_j)^{-1} \pmod{q}$ when $b_i - b_j$ is invertible. Time $O(\sqrt{q})$ group operations via the birthday bound, and $O(1)$ memory (Floyd cycle-finding, no table). It uses only the group operation and the group order, so it applies to **any** group — including elliptic-curve groups — and is the benchmark behind “ECC strength $\approx \sqrt{q}$ ”. The analogous factoring variant runs in $O(\sqrt{p})$ where p is the smallest prime factor: on the guide’s example $n = 8051$, the Floyd trace reaches $\gcd(|x - y|, n) = 97$ at step 3, yielding $8051 = 97 \cdot 83$ with cost governed by the smaller factor.

Index Calculus. A *special-purpose* method exploiting how elements of $\mathbb{Z}_p^*$ are represented as integers with a smooth factorization over a small base $\mathcal{B} = \{p_1, \dots, p_t\}$: collect smooth values $g^r = \prod_i p_i^{e_i}$, turn them into linear equations $r \equiv \sum_i e_i \log_g p_i \pmod{p-1}$, solve for the base logs, then combine h with powers of g until another smooth value yields $\log_g h$. Its cost is **subexponential**, $L_p[1/2, \sqrt{2}] = \exp((\sqrt{2} + o(1))(\ln p)^{1/2}(\ln \ln p)^{1/2})$, which beats the $O(\sqrt{q})$ generic bound for large p . Memory is likewise subexponential: it must store the collected relations and solve a linear system over $\mathbb{Z}_{p-1}$.

Applicable groups. Index Calculus needs the smooth-factorization structure, so it works in $\mathbb{Z}_p^*$ (and finite fields $\mathbb{F}_{p^k}$) but has **no analogue in elliptic-curve groups**, because curve points have no notion of factoring over small primes. That asymmetry is exactly why finite-field DH/DSA need huge moduli (L in the size table: 2048–15360 bits) while ECC gets the same strength from much smaller keys: for well-chosen curves the best known attack is generic Pollard–rho, $O(\sqrt{q})$.

	Time	Memory	Groups
Pollard–rho	$O(\sqrt{q})$, generic	$O(1)$	any group (incl. ECC)
Index Calculus	subexponential $L_p[1/2, \sqrt{2}]$	subexponential (relations + linear algebra)	$\mathbb{Z}_p^*/$ finite fields, not ECC

4.31 Q26. Why does ECC use smaller keys for the same classical strength?

Key size is dictated by the *best known attack*, and the two settings sit in different complexity regimes.

- For RSA and finite-field DLP, the best classical algorithms are **subexponential** (index calculus / number-field sieve). Their cost grows like $L_N[1/3, c]$, so the modulus must grow very fast to buy each extra bit of security: 128-bit classical strength already needs a 3072-bit RSA modulus or a 3072-bit finite-field L (with a 256-bit subgroup order N).
- For a well-chosen elliptic curve with a large prime-order subgroup, no subexponential attack is known. The group is generic — its points admit no smoothness structure — so the best known classical attack is Pollard–rho in $O(\sqrt{q})$ group operations. Security is therefore about $\frac{1}{2} \log_2 q$ bits, i.e. **linear** in the key size.

Consequence. To reach λ bits of classical security, ECC needs a curve order $q \approx 2^{2\lambda}$, whereas RSA needs an n with 2λ bits *only if* the subexponential attack behaved linearly — which it does not. Hence the guide’s table: at 128 bits of strength, ECC needs only 256–383 bits versus 3072 bits for RSA/DH. Smaller keys mean faster arithmetic, smaller signatures/certificates, and less bandwidth, with roughly the same demonstrated hardness. Caveat: this comparison is for *classical* security; a large-scale quantum computer would break RSA, DH and ECC alike (Shor), so the classical advantage is not a post-quantum one.

4.31.1 Hashes and authentication

4.32 Q27. Preimage, second-preimage and collision resistance

Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ be a hash function. The three properties differ in **what the adversary is given** and **what it must produce**.

- **Preimage resistance (one-wayness).** Given a digest y , it is infeasible to find any x with $H(x) = y$. Formally, for every PPT $\mathcal{A}$,

$$\Pr_y [H(\mathcal{A}(y)) = y] \leq \text{negl}(\lambda),$$

where y is sampled as $H(x)$ for a random x . The **generic attack** is brute force: about 2^n evaluations, since a random guess hits the fixed y with probability 2^{-n} .

- **Second-preimage resistance (weak collision resistance).** Given a **fixed** input x , it is infeasible to find a different $x' \neq x$ with $H(x') = H(x)$. The generic attack is again about 2^n evaluations: the target digest $H(x)$ is already fixed before the adversary starts, so it cannot search a set of its own.
- **Collision resistance (strong collision resistance).** It is infeasible to find **any** pair $x \neq x'$ with $H(x) = H(x')$. Now the adversary picks **both** messages, so it can use the birthday paradox: after about $2^{n/2}$ evaluations a collision among the computed digests is likely.

The key conceptual point is the ordering. In a second-preimage attack one of the two colliding messages is chosen by the challenger and frozen; in a collision attack the adversary chooses both and searches for any match. Because the adversary in the collision game has strictly more freedom, **collision resistance is the stronger requirement**, and it implies second-preimage resistance whenever the set of valid inputs is much larger than the image (a fixed x has few preimages, so any collision pair is generically a second preimage). The converse does **not** hold: there are functions that are preimage-resistant yet trivially collision-broken (e.g. a function that ignores the last bit of the input). Preimage resistance is independent of the other two in general.

4.33 Q28. Derivation of the birthday approximation

Sample q values $x_1, \dots, x_q$ uniformly and independently from a set of size N . A collision is $x_i = x_j$ for some $i < j$. Instead of counting collisions directly, compute the probability of the complement, “all distinct”, by conditioning on each new draw:

$$\Pr[\text{all distinct}] = \prod_{x_1} \left(1 - \frac{1}{N}\right) \cdot \left(1 - \frac{2}{N}\right) \cdots \left(1 - \frac{q-1}{N}\right) = \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right).$$

Hence the exact collision probability is

$$\Pr[\text{collision}] = 1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right).$$

For $q \ll N$ each factor satisfies $\ln(1 - i/N) = -i/N + O(i^2/N^2)$, and because the i are at most q the accumulated second-order term is $O(q^3/N^2)$, negligible, so

$$\ln \Pr[\text{all distinct}] = \sum_{i=0}^{q-1} \ln \left(1 - \frac{i}{N}\right) \approx - \sum_{i=0}^{q-1} \frac{i}{N} = - \frac{q(q-1)}{2N}.$$

Exponentiating:

$$\Pr[\text{collision}] \approx 1 - \exp\left(-\frac{q(q-1)}{2N}\right),$$

which is the standard form $\approx q(q-1)/(2N)$ when the exponent is small (the union-bound/first-moment estimate $\binom{q}{2}/N$). Setting the probability to $1/2$ gives $\exp(-q(q-1)/(2N)) = \frac{1}{2}$, i.e. $q(q-1) = 2N \ln 2$, and dropping the linear term:

$$q \approx \sqrt{2N \ln 2} \approx 1.177\sqrt{N}.$$

Numerically with $N = 36500$: $\sqrt{2 \cdot 36500 \cdot \ln 2} \approx 224.94$. The exact product gives $\Pr[225] \approx 0.49934$ and $\Pr[226] \approx 0.50243$, so the smallest q reaching probability $1/2$ is $q = 226$ (both numbers computed with Python). The lesson for cryptography: collisions in an N -element space need only about $\sqrt{N}$ samples.

4.34 Q29. Why a 256-bit output gives about 128 bits against collisions

Let the digest length be $n = 256$, so the image has $N = 2^n = 2^{256}$ possible values. By Q28 a collision is found with probability about $1/2$ after

$$q \approx \sqrt{2N \ln 2} = \sqrt{2 \cdot 2^{256} \ln 2} \approx 1.177 \cdot 2^{128} \approx 2^{128.24}$$

evaluations of H . In exponent language, $q \approx 2^{n/2}$, so the search cost is 2^{128} , i.e. the **effective security is $n/2 = 128$ bits**, only half of the output length. Concretely, $2^{128} \approx 3.40 \times 10^{38}$ operations is infeasible today, whereas 2^{256} is astronomically out of reach. This is exactly why a function intended to resist collisions should have an output of at least 256 bits when 128 bits of collision security are wanted; the birthday bound caps the achievable security at half the output size. It is worth stressing that only the **collision** property suffers this halving: preimage and second-preimage resistance still cost about 2^{256} , because there the adversary is targeting a fixed digest and cannot search its own birthday set. This is why MD5 (128-bit, collision security only 2^{64}) and SHA-1 (160-bit, about 2^{80}) are considered broken for collision-sensitive uses such as signatures.

4.35 Q30. Why $H(k\|m)$ must not be adopted as a MAC without analysis

A MAC must authenticate a message under a secret key, so a first, naive proposal is

$$t = H(k\|m),$$

with k secret and m the message. The problem is that “hash” alone says nothing about keyed authentication; security must be **proved** for the construction, and this one fails for the dominant Merkle–Damgård family (MD5, SHA-1, SHA-2). Those hashes process the input in blocks and output the chaining state after a final length-padding block, i.e. the output is exactly the internal state

$$t = \text{state}_k = H(k\|m).$$

Because the digest reveals that state, an adversary who knows the length of k (a very mild assumption, and often known exactly) can continue the computation **without knowing k** : it appends the padding block used by the hash, then appends any extra block m' , and recomputes

$$H(k\|m\|\text{pad}\|m') = f(\text{state}_k, \text{new block}),$$

where f is the public compression function. The result is a valid tag for the **new** message $m\|\text{pad}\|m'$, which the adversary never queried. This is the **length-extension attack**: a selective forgery from only one known message–tag pair, which breaks even the weakest MAC notion. A second, independent issue is **ambiguity of concatenation**: $H(k\|m)$ gives the hash no way to tell where k ends and m begins, so different (k, m) splits can produce the same input. Because of these attacks the naive construction must not be used; the correct approach is a construction with a proof or a sound design, such as HMAC, which applies the hash twice with the inner/outer padding constants

$$\text{HMAC}_K(m) = H((K' \oplus \text{opad})\|H((K' \oplus \text{ipad})\|m)),$$

so that extension of the inner digest does not yield a valid outer tag. The general rule: a construction being “obvious” does not make it secure; $H(k\|m)$ is not automatically a MAC.

4.36 Q31. MAC versus signature: who can verify and who can forge

Both primitives provide integrity and authenticity, but the **distribution of secrets** changes who is trusted.

- **MAC** (symmetric): a single shared key k is generated by Gen and given to both the sender and the verifier. The tag is $t = \text{Mac}_k(m)$ and $\text{Vrfy}_k(m, t) \in \{0, 1\}$. Because verification uses the same secret k ,

anyone who can verify can also forge: given k , the verifier can compute $\text{Mac}_k(m')$ for any m' . There is no distinction between “the one who authenticates” and “the one who checks”. Consequently a MAC does **not** prove authorship to a third party — if Alice and Bob share k , a tag proves only that *one of them* (or someone holding k) produced it, so Bob cannot convince a judge that Alice, and not Bob himself, sent the message. Formally, unforgeability under chosen-message attack says a PPT adversary with oracle access to tags cannot output a valid tag for a new message, but the symmetric verifier is trivially outside that model.

- **Digital signature** (asymmetric): $\text{Gen}(1^\lambda)$ outputs a private signing key sk and a public verification key pk . Only the holder of sk can produce $\sigma = \text{Sign}_{\text{sk}}(m)$; **anyone** can check $\text{Vrfy}_{\text{pk}}(m, \sigma)$ with the public key. Thus the set of verifiers is unbounded and disjoint from the set of signers: a public verifier must **not** be able to sign. This yields **public verifiability** and, when keys and identity are properly managed, **non-repudiation**: the signer cannot later deny having signed, because no one else could have produced the signature. EUF-CMA again captures unforgeability: after obtaining signatures of chosen messages, the adversary cannot sign a new message.

In short: MAC = shared secret, verifier can also forge, no third-party proof, usually faster; signature = key pair, verifier cannot forge, public verifiability, usually more expensive. A caveat from the guide: non-repudiation is not produced by the equation alone — it also depends on key protection, identity/certification, policy and legal context.

4.37 Q32. Two forgeries against textbook RSA signatures

Textbook RSA signs by $\sigma = m^d \bmod n$ and verifies by $\sigma^e \stackrel{?}{\equiv} m \pmod{n}$, with $n = pq$ and $ed \equiv 1 \pmod{\varphi(n)}$. We use the guide’s key $n = 3233$, $e = 17$, $d = 2753$ (here $n = 61 \cdot 53$ and $\varphi(n) = 60 \cdot 52 = 3120$; indeed $17 \cdot 2753 \bmod 3120 = 1$).

(a) **Existential forgery by verifying backward.** The verifier accepts any pair (σ, m) with $\sigma^e \equiv m \pmod{n}$. So an adversary simply picks an arbitrary σ , sets $m = \sigma^e \bmod n$, and publishes (m, σ) without ever knowing d . Take $\sigma = 2$:

$$m = 2^{17} \bmod 3233 = 1752,$$

and indeed $1752^d \bmod 3233 = 2 = \sigma$, so $(1752, 2)$ verifies. The attacker has produced a valid signature for a message it did not choose (an existential forgery). It cannot target a prescribed message, but it already violates the minimal security notion. In practice a countermeasure is to require that the message have a valid, redundant encoding (hash-and-sign with padding, e.g. RSA-PSS), so that a random σ is overwhelmingly unlikely to map to a well-formed encoding.

(b) **Multiplicative malleability.** Signatures are multiplicative:

$$\sigma_1 \sigma_2 \equiv m_1^d m_2^d = (m_1 m_2)^d \pmod{n},$$

so from two valid signatures one obtains a valid signature on the product $m_1 m_2 \bmod n$. Using the guide’s example $\sigma_1 = 65^d \bmod 3233 = 588$, and taking $m_2 = 11$ with $\sigma_2 = 11^d \bmod 3233 = 804$, we have

$$m_1 m_2 = 65 \cdot 11 \bmod 3233 = 715, \quad \sigma_1 \sigma_2 = 588 \cdot 804 \bmod 3233 = 734,$$

and verification confirms $734^{17} \bmod 3233 = 715$, so $(\sigma_1 \sigma_2)$ is a valid signature on the new message 715 that was never signed (all values verified with Python). The adversary again produces a signature for an unintended message by algebraically combining known signatures. Both attacks are avoided by hashing and encoding the message before exponentiating (RSA-PSS), so the multiplicative structure no longer acts on the message itself; merely replacing m by $H(m)$ without analyzing the encoding and the security model does not automatically justify security.

4.38 Q33. Demonstrating the ElGamal signature verification equation

ElGamal signatures work in $\mathbb{Z}_p^*$ with p prime and g a generator. The secret is $x \stackrel{\$}{\leftarrow} \{1, \dots, p-2\}$ and the public key is $y = g^x \bmod p$. To sign a digest $h = H(m)$, choose $k \in \mathbb{Z}_{p-1}^*$ (so $\gcd(k, p-1) = 1$), compute the nonce commitment $r = g^k \bmod p$, and set

$$s \equiv k^{-1}(h - xr) \pmod{p-1}.$$

The signature is (r, s) and verification checks

$$g^h \stackrel{?}{\equiv} y^r r^s \pmod{p}.$$

Why it works. Substitute $y = g^x$ and $r = g^k$:

$$y^r r^s \equiv g^{xr} g^{ks} = g^{xr+ks} \pmod{p}.$$

The defining relation of s gives $ks \equiv h - xr \pmod{p-1}$, i.e. $xr + ks \equiv h \pmod{p-1}$. Since g has order dividing $p-1$, exponents may be reduced modulo $p-1$:

$$g^{xr+ks} = g^h.$$

So the two sides of the verification equation coincide whenever s is computed correctly.

Small numeric case (guide's parameters). $p = 23$, $g = 2$, $x = 6$, so $y = 2^6 \bmod 23 = 18$. Sign $h = 7$ with $k = 5$; $\gcd(5, 22) = 1$, and $k^{-1} \equiv 9 \pmod{22}$ since $5 \cdot 9 = 45 \equiv 1 \pmod{22}$. Then

$$r = 2^5 \bmod 23 = 9, \quad s \equiv 9(7 - 6 \cdot 9) = 9 \cdot (-47) \equiv 9 \cdot 19 = 171 \equiv 17 \pmod{22},$$

so the signature is $(r, s) = (9, 17)$. Verification computes

$$y^r r^s \bmod p = 18^9 \cdot 9^{17} \bmod 23 = (12 \cdot 3) \bmod 23 = 13, \quad g^h \bmod p = 2^7 \bmod 23 = 13,$$

and the two sides are equal, so it accepts (all values verified with Python). Two remarks the professor expects: s is reduced modulo $p-1 = 22$, **not** modulo $p = 23$ — confusing the two moduli is the classic error; and the nonce k must stay secret, unpredictable and never reused.

4.39 Q34. Recovery of k and x when DSA reuses a nonce

In DSA over a subgroup of prime order q ($q \mid p-1$, and g an element of order q in $\mathbb{Z}_p^*$), a signature on $H(m)$ with nonce $k \in \mathbb{Z}_q^*$ is

$$r = (g^k \bmod p) \bmod q, \quad s = k^{-1}(H(m) + xr) \bmod q,$$

where $x \in \mathbb{Z}_q^*$ is the private key and $y = g^x \bmod p$ is public. Suppose the signer is careless and uses the **same** k to sign two different messages m_1, m_2 (digests h_1, h_2). Then the commitment $r = (g^k \bmod p) \bmod q$ is identical in both signatures, which is the tell-tale sign of reuse. The two equations are

$$s_1 \equiv k^{-1}(h_1 + xr) \pmod{q}, \quad s_2 \equiv k^{-1}(h_2 + xr) \pmod{q}.$$

Recover k . Multiply both by k and subtract the second from the first; the term xr cancels:

$$k(s_1 - s_2) \equiv h_1 - h_2 \pmod{q} \implies k \equiv (h_1 - h_2)(s_1 - s_2)^{-1} \bmod q,$$

which is defined because $s_1 \neq s_2$ as long as $h_1 \not\equiv h_2 \pmod{q}$; subtracting the equations is the step that eliminates the unknown key. **Recover x .** Once k is known, either signing equation is linear in x ; from the first,

$$s_1 k \equiv h_1 + xr \pmod{q} \implies x \equiv (s_1 k - h_1) r^{-1} \bmod q,$$

which is the formula in the guide. It is valid because $r \neq 0 \bmod q$ by definition of a valid DSA signature.

Small numeric case (guide's parameters). $p = 23$, $q = 11$, $g = 2$ of order 11, $x = 3$, so $y = 2^3 \bmod 23 = 8$. Reuse $k = 5$ (with $k^{-1} \equiv 9 \pmod{11}$) on digests $h_1 = 7$ and $h_2 = 4$:

$$r = (2^5 \bmod 23) \bmod 11 = 9, \\ s_1 \equiv 9(7 + 3 \cdot 9) = 9 \cdot 34 = 306 \equiv 9 \pmod{11}, \quad s_2 \equiv 9(4 + 3 \cdot 9) = 9 \cdot 31 = 279 \equiv 4 \pmod{11}.$$

Both signatures $(r, s_1) = (9, 9)$ and $(r, s_2) = (9, 4)$ verify against $y = 8$ (checked with the standard verification: $w = s^{-1} \bmod q$, $u_1 = hw$, $u_2 = rw$, $v = (g^{u_1} y^{u_2} \bmod p) \bmod q$, giving $v = 9 = r$ in both cases). The attacker sees $r = 9$ twice, so it guesses nonce reuse and computes

$$k \equiv (h_1 - h_2)(s_1 - s_2)^{-1} \equiv (7 - 4)(9 - 4)^{-1} \equiv 3 \cdot 5^{-1} \pmod{11}.$$

Since $5^{-1} \equiv 9 \pmod{11}$ (as $5 \cdot 9 = 45 \equiv 1$), $k \equiv 3 \cdot 9 = 27 \equiv 5 \pmod{11}$, recovering the nonce exactly. Then

$$x \equiv (s_1 k - h_1) r^{-1} \equiv (9 \cdot 5 - 7) \cdot 9^{-1} \pmod{11}.$$

Now $9 \cdot 5 = 45 \equiv 1 \pmod{11}$ and $1 - 7 = -6 \equiv 5 \pmod{11}$, while $r^{-1} = 9^{-1} \equiv 5 \pmod{11}$ (since $9 \cdot 5 = 45 \equiv 1$), so $x \equiv 5 \cdot 5 = 25 \equiv 3 \pmod{11}$, recovering the true private key $x = 3$ (all steps verified with Python). A single repeated nonce therefore leaks k and then the entire private key; this is exactly the failure behind the Sony PS3 ECDSA break and the risk of biased or low-entropy nonces. The defense is a unique, unpredictable k per signature, ideally derived deterministically and verifiably (RFC 6979), and the same logic applies verbatim to ECDSA.

4.40 References by topic

Topic	Katz–Lindell, 3rd ed.
perfect secrecy	Ch. 2
computational security, PRG, CPA, and modes	§§3.1–3.6
MACs	Ch. 4
GCM/authentication	Ch. 5
hashes and birthday	Ch. 6 and Appendix A.4
Feistel, DES, and AES	§§7.2.2–7.2.5
groups, primes, RSA, and ECC	Ch. 9
Pollard, DLP, Index Calculus, and sizes	Ch. 10
Diffie–Hellman	§11.3
ElGamal and RSA/OAEP	§§12.4–12.5
RSA, DSA, and ECDSA signatures	§§13.4–13.5
Euclid and modular exponentiation	Appendix B

Complementary official references:

- NIST FIPS 197, *Advanced Encryption Standard (AES)*: <https://doi.org/10.6028/NIST.FIPS.197-upd1>
- NIST SP 800-57 Part 1 Rev. 5, *Recommendation for Key Management*: <https://doi.org/10.6028/NIST.SP.800-57pt1r5>
- NIST, policy on hash functions: <https://csrc.nist.gov/projects/hash-functions/nist-policy-on-hash-functions>